Руководство по разработке шаблонов оформления

Практическое руководство для разработчика, который создаёт внешний шаблон оформления (ZIP) для конструктора SCORM-тестов: что такое шаблон и как он живёт внутри сервиса, как за полчаса собрать первый рабочий шаблон, что положить в архив, как устроены манифест и макеты, какой синтаксис понимает движок, какие данные доступны на каждом экране, как пройти валидацию и проверку работоспособности и как загрузить шаблон в систему.

Руководство рассчитано на человека, который умеет писать простой HTML и CSS, но о сервисе узнал только что. Начните с разделов **«Введение»** и **«Быстрый старт»** — они дают целостную картину и проводят через сборку первого шаблона от пустой папки до загрузки. Разделы 1–16 — подробный справочник: к ним удобно возвращаться за деталями.

Статус: актуально; соответствует спецификации формата версии 3.16.0 и эталонному шаблону «Стандартный». Учтены системные узлы раздела (`review` , `section-results`), варианты стартового экрана (`start.*` со свойством страницы `image`), страница отчёта о результатах (`report` / `report.adaptive` , §7.3a) с постраничной раскладкой и меткой `data-page-break` , печать авторского текста с форматированием (`{{& путь }}` над парными полями `textHtml` / `textsHtml`), признак назначения поля отчёта (`settings[].scope`), вводный блок итогов (`result.introHtml`), надписи интерфейса итогов и порядок подблоков (`labels[]` , `resultsBlockOrder` , §3.4-3.5), разрез результата в карточке темы (`breakdown[]` , §7.3), блоки разделов со счётчиком (`topicGroups[]` , `ungroupedTopics[]`) и сводный разрез по тесту отдельным подблоком итогов (`result.breakdown[]` , `isBreakdown`), документ отчёта из блоков (`kind: "report.block"` , `reportDocument` , §7.3b), толкование темы и подтемы (`interpretationHtml` , признаки раскладки `hasAside` и `hasBreakdownNotes`) и заголовки итога свойствами варианта «Итоги теста» (`headingDocument` / `headingPassed` / `headingFailed`), кликабельные в PDF ссылки `<a href>` авторского текста, окраска полос подтем по выбору автора (`breakdownBarFill` , `barFill`), признак «выход закрывает раздел» на старте (`course.closesOnLeave`), условие прохождения на обложке и во введении раздела (`course.passCondition` , `sectionIntro.passCondition` , `isRequired` , `timerWarning`) и тумблер этой строки у варианта введения (`passConditionShown`).

Версия руководства: 3.16.0 · **соответствует спецификации формата:** 3.16.0 · **дата актуализации:** 2026-10-16

Руководство написано под конкретную версию спецификации (см. поле выше). При изменении контракта формата спецификация бампится и получает запись в своей «Истории версий», после чего это соответствие обновляется здесь.

Связанные документы:

- Платформа SCORM-шаблонов — формальная спецификация формата (источник истины), версия 3.16.0.

Эталонный шаблон, на который опираются примеры ниже, лежит в репозитории: `server/scorm/templates/default/`. Он проходит валидацию и проверку работоспособности, поэтому его файлы — самый надёжный образец.

Введение: что такое шаблон и как он работает

Что делает сервис

Сервис — это конструктор тестов. Автор (преподаватель, методист) собирает тест: пишет вопросы, задаёт проходной балл, порядок прохождения, лимит времени. Готовый тест сервис умеет выгружать в виде **SCORM-пакета** — это ZIP-архив с HTML, CSS и JS внутри, который загружают в систему дистанционного обучения (LMS): она показывает тест ученику и сохраняет результат.

Автор отвечает за *содержание* (какие вопросы, какие тексты, сколько баллов). За то, как всё это *выглядит и ведёт себя на экране* ученика, отвечает **шаблон оформления**. Это и есть то, что вы разрабатываете.

Что такое шаблон оформления

Шаблон — это ZIP-архив с вёрсткой и стилями экранов, которые видит ученик: стартовая страница, экран вопроса, учебные страницы, итоги. По смыслу это как «тема» (theme) для сайта или «скин» для приложения: содержание одно и то же, а внешний вид задаёт шаблон. Один и тот же тест, выгруженный с разными шаблонами, выглядит совершенно по-разному, но состоит из одних и тех же вопросов и правил.

Важно, чего в шаблоне **нет**: логики оценивания, подсчёта баллов, проверки ответов, навигации между вопросами, работы со SCORM. Всё это — задача *ядра* (движка сервиса). Шаблон только рисует то, что ядро ему передаёт. Поэтому шаблон нельзя «сломать» так, чтобы тест посчитался неверно: вы отвечаете за внешний вид, а не за арифметику.

Разделение ответственности: вы и ядро

Делает ЯДРО (не ваша забота)	Делаете ВЫ в шаблоне
Считает баллы, решает «пройдено/нет»	Показываете готовый процент и статус
Проверяет ответы, ведёт навигацию	Верстаете экран вопроса и кнопки
Хранит прогресс, пишет в SCORM	Рисуете полосу прогресса из данных
Готовит все тексты и числа (контекст)	Расставляете их по экрану
Вставляет варианты ответа, медиа	Даёте им место — «слот» — в разметке

Ключевая мысль: **вы не пишете бизнес-логику, вы оформляете готовые данные**. Ядро передаёт шаблону «контекст рендера» — набор уже посчитанных значений (название теста, номер вопроса, процент результата, готовый CSS-класс статуса), а шаблон расставляет их по своей вёрстке. Список всех доступных значений — в разделе «Публичный контекст рендера» (§7).

Один шаблон — два плеера

Тест показывается в двух местах, и оба рисуют экраны **одним и тем же движком** (`renderScreenInto`):

- **SCORM-пакет** — то, что уходит в LMS и видит ученик.
- **Веб-предпросмотр** внутри сервиса — автор смотрит тест прямо в браузере, там же работает предпросмотр вашего шаблона в реестре.

Отдельной вёрстки «для веба» и «для SCORM» не существует. Вы пишете разметку один раз — она одинаково работает на обоих хостах. Что видно в предпросмотре, то увидит и ученик.

Экраны, из которых состоит прохождение

Шаблон описывает несколько типов экранов. Не все обязательны: если шаблон какой-то экран не объявил, ядро возьмёт его из встроенного «Стандартного» шаблона.

Экран	Когда показывается
Старт	Лендинг перед началом: название, число вопросов, кнопка «Начать»
Вопрос	Один вопрос с вариантами ответа
Учебная страница	Текст/картинка между вопросами (введение, материал)
Итоги теста	Финальный результат: процент, «пройдено/нет», разбивка по темам
Обзор / Итоги раздела	Служебные экраны для тестов с разделами (можно не делать сразу)

Минимум, с которого стоит начать, — четыре первых экрана. Остальное добавляется позже; пока шаблон их не объявил, они берутся из «Стандартного».

Кроме экранов шаблон может описать **отчёт о результатах** — страницу, которую ученик скачивает файлом PDF с экрана итогов. Это не экран прохождения: у неё своя вёрстка, свои правила стилей и своя проверка, поэтому она разобрана отдельно (§7.3a). Отчёт тоже необязателен: не объявили — ученик получит его по макету «Стандартного», но с цветами и логотипом вашего шаблона.

Как ядро рисует один экран

Экран собирается из двух кусков: **оболочки** (`shell.html`, общий каркас — рамка, полоса прогресса) и **макета** текущего экрана (например `layouts/start.html`), который ядро вставляет внутрь оболочки. Дальше ядро проходит по разметке макета и подставляет данные четырьмя

способами (порядок важен):

1. **`{{ путь }}`** и родственные `{{#if}}` / `{{#each}}` — «мустачные» вставки: подставляют текст и включают/повторяют блоки.
2. **`data-path="course.title"`** — то же, что `{{ }}`, но как атрибут: в элемент подставляется текст по пути. Удобно оставлять «заглушку» в статической вёрстке.
3. **`data-slot="имя"`** — область, куда ядро само вставляет готовый HTML (текст вопроса, варианты ответа, контент страницы). Вы лишь отводите место.
4. **`data-placeholder="ключ"`** — на учебных страницах: сюда ядро кладёт значение, которое ввёл автор (заголовок, текст).

Крошечный пример. Макет:

```
<h1 data-path="course.title"></h1>
<p>Вопросов: {{ course.questionCount }}</p>
{{#if course.timeLimitLabel}}<p>Время: {{ course.timeLimitLabel }}</p>{{/if}}
```

Контекст, который дало ядро: `course.title = "Основы ИБ"`, `course.questionCount = 10`, `course.timeLimitLabel = "30 мин"`. Результат на экране:

```
<h1>Основы ИБ</h1>
<p>Вопросов: 10</p>
<p>Время: 30 мин</p>
```

Если бы `timeLimitLabel` было пустым, третий абзац просто не появился бы. Весь текст экранируется автоматически — вставить исполняемый HTML через `{{ }}` нельзя (это защита; подробности — §5).

Мини-словарь

Термин	Что значит
Ядро (core)	Движок сервиса: считает, проверяет, готовит данные, рисует экран вашим шаблоном
Шаблон оформления	Ваш ZIP: вёрстка и стили экранов ученика
Манифест	`manifest.json` — «паспорт» шаблона: что в нём есть и где лежит
Макет (layout)	HTML-фрагмент одного экрана (`layouts/start.html`)
Оболочка (shell)	Общий каркас `shell.html` , в который монтируются экраны
Контекст рендера	Готовые данные от ядра (`course.*` , `result.*` , …), которые вы расставляете
Слот (slot)	Область `data-slot="…"` , которую заполняет ядро (варианты ответа, контент)
Placeholder	Поле контентной страницы, которое заполняет автор теста
DSL	Мини-язык вставок `{{ … }}` (подмножество mustache)
contentTemplate	Описание «типа» страницы в манифесте (какие поля у неё есть)
kind	Функциональный вид экрана: `start` , `questions` , `info` , `results` , …

Быстрый старт

Есть два пути. Оба приведут к рабочему шаблону; выбирайте по ситуации.

- **Путь 1 — изменить готовый.** Самый быстрый и надёжный способ: скачать «Стандартный» шаблон, поменять цвета и вёрстку, загрузить обратно. Начатую точку дают заведомо валидные файлы.
- **Путь 2 — собрать с нуля.** Полезно, чтобы понять устройство: ниже — минимальный шаблон из нескольких файлов, который можно скопировать целиком.

Что понадобится в обоих случаях: текстовый редактор, архиватор (создать ZIP) и доступ к сервису с ролью «автор» (страница «Шаблоны»). Программировать не нужно — только HTML/CSS и немного JSON.

Путь 1. Изменить готовый шаблон

1. Откройте в сервисе **«Шаблоны»** (`/author/templates`).
2. В карточке «Стандартный» откройте меню (три точки) → **«Экспортировать ZIP»**. Скачается валидная заготовка со всеми экранами.

3. Распакуйте архив. Внутри — та же структура, что описана ниже (`manifest.json`, `shell.html`, `layouts/`, `styles/`, …).
4. В `manifest.json` **обязательно смените** `id` (иначе загрузка отклонится с `ID_EXISTS`) и `name`, при желании — `version`:

   ```
   {
     "id": "my-first-template",
     "name": "Мой первый шаблон",
     "version": "1.0.0"
   }
   ```

5. Начните с малого: поменяйте цвета в `styles/theme.css` (см. §10) и, например, заголовок стартового экрана в `layouts/start.html`.
6. Запакуйте папку обратно в ZIP (см. «Шаг 11» ниже — важно, чтобы `manifest.json` был в корне) и загрузите: **«Шаблоны»** → **«Загрузить шаблон»**.
7. Откройте карточку, посмотрите экраны, нажмите **«Проверить работоспособность»**, затем **«Активировать»** (см. «Шаг 12»).

Дальше меняйте по одному экрану за раз, каждый раз перепроверяя. Полное описание файлов — в справочнике ниже; здесь же — путь 2, который показывает те же файлы «с нуля».

Путь 2. Собрать минимальный шаблон с нуля

Ниже — законченный минимальный шаблон. Скопируйте файлы как есть — он проходит валидацию и показывает все четыре основных экрана. Это ваша песочница: меняйте по кусочку и сразу смотрите результат в предпросмотре.

Шаг 1. Дерево файлов

Создайте папку и такие файлы:

```
my-first-template/
  manifest.json
  shell.html
  layouts/
    start.html
    question.html
    content.html
    results.html
  styles/
    theme.css
    base.css
  demo/
    course.json
  preview.svg
```

Шаг 2. manifest.json

«Паспорт» шаблона: перечисляет экраны, поля страниц, параметры оформления и демо-данные. Скопируйте целиком:

```
{
  "id": "my-first-template",
  "name": "Мой первый шаблон",
  "version": "1.0.0",
  "templateApiVersion": "1.0",
  "description": "Минимальный одноколоночный шаблон — стартовая точка для своего.",
  "params": [
    {
      "key": "primaryColor",
      "type": "color",
      "label": "Основной цвет",
      "description": "Кнопки и выбранный вариант ответа.",
      "default": "217 91% 45%",
      "group": "Цвета",
      "section": "branding"
    }
  ],
  "contentTemplates": [
    { "key": "start.basic", "label": "Старт", "kind": "start", "pageKind": "start",
"isDefault": true, "placeholders": [] },
    { "key": "question.basic", "label": "Вопрос", "kind": "questions", "isDefault": true,
"placeholders": [] },
    {
      "key": "info.basic",
      "label": "Учебная страница",
      "kind": "info",
      "pageKind": "content.info",
      "isDefault": true,
      "placeholders": [
        { "key": "title", "type": "text", "label": "Заголовок", "maxLength": 120 },
        { "key": "body", "type": "richText", "label": "Текст" }
      ]
    },
    { "key": "results.basic", "label": "Итоги теста", "kind": "results", "pageKind":
"results", "isDefault": true, "placeholders": [] }
  ],
  "layouts": {
    "shell": "shell.html",
    "start": "layouts/start.html",
    "content": "layouts/content.html",
    "question": "layouts/question.html",
    "results": "layouts/results.html"
```

```
  },
  "assets": {
    "styles": ["styles/theme.css", "styles/base.css"],
    "scripts": [],
    "images": [],
    "preview": "preview.svg"
  },
  "capabilities": {
    "navigation": ["linear", "locked"],
    "progress": ["questions", "pages", "hidden"],
    "timer": true,
    "questionTypes": ["single", "multiple", "matching", "ranking"],
    "runtimeApi": "1.0"
  },
  "preview": {
    "demoData": "demo/course.json",
    "defaultRoute": "start",
    "routes": [
      { "route": "start", "label": "Старт" },
      { "route": "question.single", "questionId": "demo-q1", "label": "Вопрос" },
      { "route": "content.info", "templateKey": "info.basic", "label": "Учебная страница"
},
      { "route": "results", "label": "Итоги" }
    ]
  }
}
```

Что здесь важно на первый раз:

- `id` — латиница/цифры/дефис, уникален в системе; `version` — вида `1.0.0`.
- `contentTemplates` перечисляет типы экранов; `kind` задаёт роль (`start`, `questions`, `info`, `results`). У учебной страницы объявлены два поля (`placeholders`) — их заполнит автор теста.
- `layouts` связывает ключ экрана с файлом макета.
- `preview.routes` — экраны, которые показываются в предпросмотре и проверяются; для каждого должны быть данные в `demo/course.json`.

Полное описание всех полей манифеста — §3.

Шаг 3. shell.html — оболочка

Общий каркас. Единственное обязательное — контейнер `data-slot="page"`, в него ядро монтирует текущий экран:

```
<div class="tpl-shell">
  <div class="tpl-progress"><div class="tpl-progress__fill" id="tb-progress-fill"></div>
</div>
  <main id="app" class="tpl-screen" data-slot="page" tabindex="-1"></main>
</div>
```

Шаг 4. Стили: theme.css и base.css

`theme.css` — только «токены» (переменные). **Цвета хранятся как HSL-компоненты** без `hsl( … )`, чтобы параметр `primaryColor` мог их переопределять (см. §10):

```
:root {
  --background: 225 12% 12%;
  --foreground: 0 0% 98%;
  --primary: 217 91% 45%;
  --card: 225 12% 18%;
  --border: 225 10% 28%;
  --muted: 225 8% 22%;
  --font-sans: Inter, -apple-system, "Segoe UI", sans-serif;
  --radius: 12px;
}
```

`base.css` — сами стили. Токены подставляются через `hsl(var(--token))`:

```
.tpl-shell {
  min-height: 100%;
  background: hsl(var(--background));
  color: hsl(var(--foreground));
  font-family: var(--font-sans);
}
.tpl-progress { height: 4px; background: hsl(var(--muted)); }
.tpl-progress__fill { height: 100%; width: 0; background: hsl(var(--primary)); transition:
width .3s; }
.tpl-screen { display: block; max-width: 760px; margin: 0 auto; padding: 32px 20px; }

.screen__title { font-size: 28px; margin: 0 0 8px; }
.screen__desc { color: hsl(var(--foreground) / .8); margin: 0 0 20px; }

.facts { list-style: none; display: flex; gap: 12px; flex-wrap: wrap; padding: 0; margin: 0
0 24px; }
.facts li { background: hsl(var(--card)); border: 1px solid hsl(var(--border)); border-
radius: var(--radius); padding: 10px 14px; }

.btn {
  appearance: none; cursor: pointer; font: inherit;
  background: hsl(var(--primary)); color: #fff; border: none;
  border-radius: var(--radius); padding: 12px 22px;
}
.btn--ghost { background: transparent; color: hsl(var(--foreground)); border: 1px solid
hsl(var(--border)); }
.actions { display: flex; gap: 12px; justify-content: flex-end; margin-top: 24px; }

.card { background: hsl(var(--card)); border: 1px solid hsl(var(--border)); border-radius:
var(--radius); padding: 20px; }
.question__meta { color: hsl(var(--foreground) / .7); margin-bottom: 8px; }
.question__text { font-size: 20px; margin-bottom: 16px; }

.results { text-align: center; }
.results__pct { font-size: 48px; font-weight: 700; }
.results__status { margin: 8px 0 20px; font-size: 20px; }
.results__status.is-pass { color: #3ad17a; }
.results__status.is-fail { color: #ff6b6b; }
.results__topics { list-style: none; padding: 0; text-align: left; }
.results__topics li { display: flex; justify-content: space-between; padding: 8px 0;
border-bottom: 1px solid hsl(var(--border)); }
```

Шаг 5. Стартовый экран (layouts/start.html)

Целиком на `data-path` и DSL — управляемых слотов здесь нет. Кнопки помечаются `data-action`, ядро само их привязывает:

```
<section class="screen start">
  <h1 class="screen__title" data-path="course.title"></h1>
  {{#if course.description}}<div class="screen__desc">{{& course.descriptionHtml }}</div>
{{/if}}

  <ul class="facts">
    <li><strong data-path="course.questionCount"></strong> вопросов</li>
    {{#if course.passPercent}}<li>проходной балл: <strong data-path="course.passPercent">
</strong>%</li>{{/if}}
    {{#if course.timeLimitLabel}}<li>время: <strong data-path="course.timeLimitLabel">
</strong></li>{{/if}}
  </ul>

  <div class="actions">
    {{#if state.canResume}}<button type="button" class="btn btn--ghost" data-
action="resume">{{state.resumeLabel}}</button>{{/if}}
    {{#if state.canStart}}<button type="button" class="btn" data-action="start-test">
{{state.startLabel}}</button>{{/if}}
  </div>
</section>
```

Описание теста — единственное место этого экрана, где печатается не строка, а разметка. Автор пишет его в одном из трёх режимов («Простой текст», «Форматированный», «HTML»), и ядро кладёт в контекст ПАРУ: `course.description` — исходный текст, `course.descriptionHtml` — готовая разметка. Печатайте разметку через `{{& … }}`, а гейт ставьте на строку.

Две вещи, на которых здесь легко ошибиться:

- **`data-path` на этом узле стоять не должен.** Проход по `[data-path]` заполняет элемент присваиванием `textContent` уже ПОСЛЕ прохода DSL — он затрёт напечатанную разметку, и описание молча покажется тегами или пустотой.
- **Элемент должен быть блочным** (`div`, а не `p`): описание может нести абзацы и списки, а `<p>` внутри `<p>` браузер закрывает сам, и вёрстка разъезжается.

Шаблон, который печатает только `course.description` через `data-path`, остаётся рабочим — он покажет описание без оформления, как показывал всегда.

Шаг 6. Экран вопроса (layouts/question.html)

Здесь важны два обязательных слота: текст вопроса и зона интерактива (варианты ответа). Их заполняет ядро — вам достаточно отвести им место. Кнопку «ответить/ далее» на экране вопроса тоже добавляет ядро, отдельной кнопки не нужно:

```
<section class="screen question">
  <div class="card">
    <div class="question__meta" data-path="state.questionCounterLabel"></div>
    <div class="question__text" data-slot="question-text"></div>
    <div data-slot="question-media"></div>
    <div data-slot="question-feedback"></div>
    <div data-slot="question-interaction"></div>
  </div>
</section>
```

Имена слотов должны быть точно `question-text` и `question-interaction` — иначе экран отрисуется из «Стандартного» шаблона (§4.2).

Шаг 7. Контентная страница (layouts/content.html)

Учебная страница (введение, материал). Ядро вставляет поля автора в слот `page-content`, а кнопку «Далее» вы помечаете `data-nav="next"`:

```
<section class="screen content">
  <div class="card" data-slot="page-content"></div>
  <div class="actions">
    {{#if page.canGoBack}}<button type="button" class="btn btn--ghost" data-
nav="prev">Назад</button>{{/if}}
    <button type="button" class="btn" data-nav="next">{{page.nextLabel}}</button>
  </div>
</section>
```

`page.nextLabel` — подпись кнопки (по умолчанию «Далее»), её отдаёт ядро (§8.1).

Шаг 8. Экран результатов (layouts/results.html)

Все значения (`passClass` , `statusLabel` , проценты) ядро уже посчитало — просто подставьте их:

```
<section class="screen results">
  <div class="results__pct"><span data-path="result.scorePercent"></span>%</div>
  <div class="results__status {{result.passClass}}" data-path="result.statusLabel"></div>

  <p>Верно <span data-path="result.correct"></span> из <span data-
path="result.totalQuestions"></span></p>

  {{#if result.topicResults}}
  <ul class="results__topics">
    {{#each result.topicResults}}
    <li class="{{passClass}}"><span>{{topicName}}</span><span>{{percent}}%</span></li>
    {{/each}}
  </ul>
  {{/if}}
</section>
```

Обратите внимание на `{{#each result.topicResults}}` — это цикл по темам; внутри доступны поля текущей темы (`topicName` , `percent`). Полный список полей результата — §7.3.

Шаг 9. Демо-данные (demo/course.json)

На этих данных строятся предпросмотр и проверка. Каждому маршруту из `preview.routes` нужны соответствующие данные:

```
{
  "schemaVersion": "1.0",
  "locale": "ru-RU",
  "params": { "primaryColor": "217 91% 45%" },
  "course": {
    "title": "Демо-тест по кибербезопасности",
    "description": "Небольшой тест, чтобы увидеть шаблон в деле.",
    "passPercent": 70,
    "timeLimitMinutes": 15,
    "maxAttempts": 3,
    "questionCount": 1,
    "topics": [{ "id": "t1", "title": "Основы", "status": "available" }],
    "contentPages": [
      {
        "id": "demo-info",
        "type": "info",
        "route": "content.info",
        "templateKey": "info.basic",
        "values": { "title": "Перед началом", "body": "<p>Читайте вопросы внимательно.</p>"
}
      }
    ],
    "questions": [
      {
        "id": "demo-q1",
        "type": "single",
        "prompt": "Какой пароль надёжнее?",
        "options": [
          { "id": "a", "text": "12345" },
          { "id": "b", "text": "Длинная случайная фраза", "correct": true }
        ]
      }
    ]
  },
  "runtime": {
    "route": "start",
    "result": {
      "scorePercent": 100,
      "passed": true,
      "status": "passed",
      "totalQuestions": 1,
      "correct": 1,
```

```
      "passClass": "is-pass",
      "statusLabel": "Пройден",
      "ringDashoffset": "0",
      "topicResults": [
        {
          "topicId": "t1",
          "topicName": "Основы",
          "percent": 100,
          "passed": true,
          "total": 1,
          "correct": 1,
          "passClass": "is-pass",
          "statusLabel": "Пройден"
        }
      ]
    }
  }
}
```

Структура демо-данных подробно разобрана в §12.

Шаг 10. Миниатюра preview.svg

Картинка карточки шаблона в реестре. Подойдёт любой маленький SVG или PNG, указанный в `assets.preview`. Минимальный SVG:

```
<svg xmlns="http://www.w3.org/2000/svg" width="320" height="200" viewBox="0 0 320 200">
  <rect width="320" height="200" fill="#1b2030"/>
  <rect x="24" y="28" width="180" height="18" rx="6" fill="#4a7dff"/>
  <rect x="24" y="60" width="272" height="10" rx="5" fill="#3a4256"/>
  <rect x="24" y="80" width="240" height="10" rx="5" fill="#3a4256"/>
</svg>
```

Сохраните как `preview.svg`.

Шаг 11. Собрать ZIP

Запакуйте **содержимое** папки так, чтобы `manifest.json` оказался в корне архива.

- Правильно: в корне ZIP лежат `manifest.json`, `shell.html`, `layouts/`, …
- Тоже сработает: всё завёрнуто в одну общую папку `my-first-template/…` — система снимет этот верхний уровень сама.
- Не сработает: `manifest.json` спрятан на два уровня вглубь.

Пути внутри архива — относительные, в прямых слэшах (`layouts/start.html`). Размер архива — до 20 МБ.

Шаг 12. Загрузить, посмотреть, проверить

1. **«Шаблоны»** → **«Загрузить шаблон»**, выберите ZIP. Пойдёт структурная проверка; при успехе появится карточка-черновик. Ошибки — с кодами (`FILE_MISSING` , `EXTERNAL_URL` , …), расшифровка в §13 и §16. Тот же ZIP принимает и раздел **«Импорт»** — единая точка загрузки файлов: он узнаёт шаблон оформления по расширению и тоже кладёт его в реестр черновиком.
2. Откройте карточку → **«Предпросмотр и проверка»**. Слева переключайте экраны (старт, вопрос, страница, итоги) — это ровно то, что увидит ученик.
3. Нажмите **«Проверить работоспособность»**: движок отрисует каждый экран на демо-данных. При успехе разблокируется активация.
4. **«Активировать»** — шаблон появится у авторов во вкладке «Оформление» теста.

Если что-то не так — раздел **«Типичные ошибки» (§16)** сопоставляет симптом с причиной.

Что дальше

Минимальный шаблон работает — дальше наращивайте по частям, сверяясь со справочником:

- **Больше типов страниц** (текст с картинкой, галерея, несколько вариантов старта): §8 и §8.1, эталонная сетка — §8.1.1.
- **Брендирование** (логотип, изображение старта, дополнительные цвета): §7.5, §9.
- **Светлая и тёмная темы**: §10.1.
- **Показатели результата** (кольцо, прогресс-бар): `resultField` в §8.
- **Все доступные данные экрана**: §7 (публичный контекст).
- **Динамика в браузере** (полоса прогресса, события): §11.
- **Перед загрузкой** пройдитесь по чек-листу §15.

Самый полный и заведомо валидный образец — встроенный «Стандартный» шаблон (`server/scorm/templates/default/`): его можно экспортировать и подсматривать решения.

---

1. Как это работает

- **Исполнение только в браузере.** Сервер никогда не исполняет HTML/CSS/JS шаблона. Он лишь читает файлы пакета, проверяет их структуру и упаковывает в SCORM. Любая динамика — это браузерный `template.js` и DSL-разметка.
- **Единый рендерер.** И SCORM-плеер, и веб-предпросмотр рисуют экраны одним и тем же движком (`renderScreenInto`). Один и тот же шаблон выглядит одинаково на обоих хостах — отдельной разметки «для веба» не существует.

- **Шаблон не знает о внутренних данных.** Макеты читают только публичный контекст рендера (`course.*`, `result.*`, `state.*`, …). Прямого доступа к внутренней модели теста у шаблона нет.
- **Никаких внешних ресурсов.** Все стили, скрипты, шрифты и картинки лежат внутри ZIP. Ссылки на CDN (`https://…`) запрещены и блокируют активацию.

Конвейер рендера одного экрана (порядок важен):

1. **DSL** — раскрываются `{{ … }}`, `{{#if}}`, `{{#each}}`, `{{> partial}}` (вывод экранируется).
2. `data-path` — в элементы с атрибутом `data-path` подставляется текст из контекста (через `textContent`, экранированно).
3. `data-slot` — в области `data-slot="name"` вставляется управляемый HTML (текст вопроса, интерактив, контент страницы) — это делает ядро.
4. `data-placeholder` — для контентных страниц заполняются области `data-placeholder="key"` по типу поля.

---

2. Структура ZIP

Минимальный пакет:

```
my-template/
  manifest.json            # обязателен, в корне
  shell.html               # внешняя оболочка плеера
  layouts/
    start.html             # стартовый экран (опционально, но рекомендуется)
    content.html           # контентные страницы (intro/info/summary/…)
    question.html          # экран вопроса
    results.html           # экран результатов
    report.html            # страница отчёта (опционально, §7.3a)
  styles/
    theme.css              # токены оформления (:root)
    base.css               # базовые стили компонентов
    report.css             # стили страницы отчёта (опционально, всё в .tb-report)
  assets/
    report/                # картинки отчёта: подложка, логотип (§7.3a)
  scripts/
    template.js            # браузерный lifecycle-скрипт (опционально)
  preview.svg              # миниатюра для карточки в реестре
  demo/
    course.json            # демонстрационные данные для предпросмотра/проверки
```

Правила упаковки:

- `manifest.json` должен оказаться в корне распакованного дерева. Если весь контент завёрнут в одну общую папку верхнего уровня (`my-template/...`), система снимет этот префикс автоматически.
- Имена путей внутри архива — относительные, в POSIX-форме (`layouts/start.html`). Абсолютные пути и выход за корень (`../`) отклоняются как небезопасные.
- Все файлы, на которые ссылается манифест, должны присутствовать. Лишние файлы допустимы, но дают предупреждение (кроме `demo/`, `preview.html`, `README`, `*.md`, `*.ejs` — они разрешены без предупреждений).

---

3. manifest.json

Манифест — единственный обязательный файл и точка входа.

3.1 Обязательные поля

Поле	Тип	Требование
`id`	string	Уникальный, шаблон `^[a-z0-9-]+$`
`name`	string	Отображаемое имя
`version`	string	Семантическая версия `\d+.\d+.\d+` (например, `1.0.0`)
`templateApiVersion`	string	Версия API платформы; поддерживается `"1.0"`
`contentTemplates`	array	Минимум один элемент
`layouts`	object	Обязательны ключи `shell`, `question`, `content`, `results`
`assets`	object	Обязателен `assets.preview`
`preview`	object	Набор демо-данных и маршрутов предпросмотра
`params`	array	Список параметров (может быть пустым `[]`)
`themes`	array	Необязательно. Палитры шаблона (§10.1); нет поля — тем нет
`labels`	array	Необязательно. Надписи интерфейса итогов (§3.4); нет поля — макет печатает свои жёсткие строки
`resultsBlockOrder`	object|array	Необязательно. Состав и порядок подблоков итогов по экранам (§3.5)
`capabilities`	object	Объявленные возможности шаблона

3.2 Полный пример манифеста

```
{
  "id": "my-template",
  "name": "Мой шаблон",
  "version": "1.0.0",
  "templateApiVersion": "1.0",
  "description": "Одноколоночный плеер с акцентным заголовком.",
  "params": [
    {
      "key": "primaryColor",
      "type": "color",
      "label": "Основной цвет",
      "default": "217 91% 42%",
      "group": "Цвета",
      "section": "branding"
    },
    {
      "key": "fontFamily",
      "type": "select",
      "label": "Шрифт",
      "default": "Inter",
      "options": ["Inter", "Roboto", "Arial"]
    },
    {
      "key": "logoUrl",
      "type": "image",
      "label": "Логотип",
      "default": null,
      "group": "Оформление",
      "section": "branding",
      "description": "Показывается в шапке экранов старта и итогов."
    }
  ],
  "contentTemplates": [
    {
      "key": "start.standard",
      "label": "Старт: стандартный",
      "kind": "start",
      "pageKind": "start",
      "isDefault": true,
      "placeholders": []
    },
```

```
  {
    "key": "start.image-right",
    "label": "Старт: изображение справа",
    "kind": "start",
    "pageKind": "start",
    "layoutFile": "layouts/start.image-right.html",
    "placeholders": [],
    "settings": [
      { "key": "image", "type": "image", "label": "Изображение" }
    ]
  },
  {
    "key": "intro.hero",
    "label": "Введение: заголовок и изображение",
    "kind": "intro",
    "pageKind": "content.intro",
    "placeholders": [
      { "key": "title", "type": "text", "label": "Заголовок", "maxLength": 120 },
      { "key": "subtitle", "type": "text", "label": "Подзаголовок", "maxLength": 300 },
      { "key": "heroImage", "type": "image", "label": "Изображение" }
    ]
  },
  {
    "key": "info.text",
    "label": "Информация: текст",
    "kind": "info",
    "pageKind": "content.info",
    "placeholders": [
      { "key": "title", "type": "text", "label": "Заголовок", "maxLength": 120 },
      { "key": "body", "type": "richText", "label": "Текст" }
    ]
  },
  {
    "key": "summary.result",
    "label": "Итог: результат",
    "kind": "summary",
    "pageKind": "content.summary",
    "placeholders": [
      { "key": "title", "type": "text", "label": "Заголовок", "maxLength": 120 },
      {
        "key": "result",
        "type": "resultField",
        "label": "Результат",
```

```
        "allowedPaths": ["result.scorePercent"],
        "defaultPath": "result.scorePercent",
        "allowedRenderers": ["core.ringChart", "core.textMetric", "core.progressBar"],
        "defaultRenderer": "core.ringChart"
      }
    ]
  },
  {
    "key": "results.standard",
    "label": "Итоги теста: стандартные",
    "kind": "results",
    "pageKind": "results",
    "placeholders": []
  },
  {
    "key": "router.menu",
    "label": "Меню карточек",
    "kind": "router",
    "pageKind": "content.router",
    "isDefault": true,
    "placeholders": [
      { "key": "title", "type": "text", "label": "Заголовок", "maxLength": 120 }
    ]
  },
  {
    "key": "question.standard",
    "label": "Стандартный макет вопроса",
    "kind": "questions",
    "isDefault": true,
    "placeholders": []
  },
  {
    "key": "report.standard",
    "label": "Отчёт: стандартный",
    "kind": "report",
    "layoutFile": "layouts/report.html",
    "styleFile": "styles/report.css",
    "isDefault": true,
    "settings": [
      {
        "key": "backgroundImage",
        "type": "image",
        "label": "Подложка страницы",
```

```
          "default": "assets/report/bg.png"
        }
      ]
    }
  ],
  "layouts": {
    "shell": "shell.html",
    "start": "layouts/start.html",
    "content": "layouts/content.html",
    "question": "layouts/question.html",
    "results": "layouts/results.html",
    "report": "layouts/report.html"
  },
  "assets": {
    "styles": ["styles/theme.css", "styles/base.css", "styles/report.css"],
    "images": ["assets/report/bg.png"],
    "scripts": ["scripts/template.js"],
    "images": [],
    "preview": "preview.svg"
  },
  "capabilities": {
    "navigation": ["linear", "locked"],
    "progress": ["questions", "pages", "hidden"],
    "timer": true,
    "questionTypes": ["single", "multiple", "matching", "ranking"],
    "runtimeApi": "1.0"
  },
  "preview": {
    "demoData": "demo/course.json",
    "defaultRoute": "start",
    "routes": [
      { "route": "start", "label": "Старт" },
      { "route": "start.image-right", "templateKey": "start.image-right", "label": "Старт:
изображение справа" },
      { "route": "content.intro", "templateKey": "intro.hero", "label": "Введение" },
      { "route": "content.info", "templateKey": "info.text", "label": "Учебный материал" },
      { "route": "question.single", "questionId": "demo-single", "label": "Вопрос: один
вариант" },
      { "route": "question.multiple", "questionId": "demo-multiple", "label": "Вопрос:
несколько" },
      { "route": "question.matching", "questionId": "demo-matching", "label":
"Сопоставление" },
      { "route": "question.ranking", "questionId": "demo-ranking", "label": "Ранжирование"
```

```
},
      { "route": "content.summary", "templateKey": "summary.result", "label": "Итог" },
      { "route": "results", "label": "Результаты" },
      { "route": "system.blocked", "label": "Доступ ограничен" }
    ]
  }
}
```

3.3 Дополнительные опциональные макеты

Кроме обязательных четырёх, можно объявить опциональные макеты-ключи:

Ключ	Экран
`start`	Стартовая страница (если нет — экран берётся из стандартного шаблона)
`results.adaptive`	Результаты адаптивного теста (уровни вместо баллов)
`section-intro`	«Введение раздела» — экран перед вопросами темы
`review`	«Обзор раздела/теста» перед завершением
`section-results`	«Итоги раздела» — вычисляемый экран после завершения раздела
`system.blocked`	Экран блокировки повторного прохождения
`system.transition`	Межуровневый переход в адаптивном тесте
`report`	Страница отчёта обычного теста (§7.3а)
`report.adaptive`	Страница отчёта адаптивного теста (§7.3а)

Эталонный `default` объявляет все перечисленные ключи. При отсутствии опционального макета соответствующий экран берётся из встроенного `default`.

Два последних ключа — парные к вариантам отчёта: сам макет отчёта резолвится по `layoutFile` ВАРИАНТА, а ключ в `layouts` нужен для деградации, когда шаблон вида не объявил.

3.4 Надписи итогов: `labels[]`

Надпись — именованная строка интерфейса, которую объявляете ВЫ с текстом по умолчанию, а автор теста может переформулировать своими словами или выключить. Заголовки «По шкалам», «Ваш результат», подписи фактов «верно», «баллов» раньше были зашиты в макет: методика, которая называет шкалы «профилями», ничего с этим сделать не могла.

Раздел необязателен. Не объявите — макет печатает свои жёсткие строки, как печатал, и подраздел «Заголовки и подписи» автору теста для вашего шаблона не показывается.

```
"labels": [
  { "key": "results.heading", "group": "Экран итогов", "label": "Заголовок итогов",
    "default": "Ваш результат" },
  { "key": "results.topics", "group": "Экран итогов", "label": "Подзаголовок тем",
    "default": "По темам", "defaults": { "report": "Результаты по темам" } },
  { "key": "topic.points", "group": "Карточка темы", "label": "Подпись баллов темы",
    "default": "Баллы" }
]
```

Поле	Обязательность	Смысл
`key`	да	Семантический ключ: по нему макет обращается к надписи (`labels.results.topics`)
`group`	да	Группировка полей в редакторе — пятнадцать надписей сплошным списком нечитаемы
`label`	да	Подпись поля для автора теста
`default`	да	Текст по умолчанию, общий для всех экранов
`defaults.<экран>`	нет	Умолчание отдельного экрана: `results`, `results.adaptive`, `section-results`, `report`

Правила, которые проверяются при активации шаблона (`LABELS_INVALID`):

- `key` уникален внутри `labels[]`, `default` у объявленной надписи обязателен;
- ключи не могут быть взаимными префиксами: `results` рядом с `results.heading` — ошибка. Контекст резолвит путь как ДЕРЕВО (`labels.results.heading` — это обход вложенных объектов), и ключ не может одновременно быть строкой-листом и веткой.

Печатается надпись из контекста, гейт стоит на самом значении — выключенная автором надпись приходит пустой строкой (см. §7.3):

```
{{#if labels.results.topics}}<h3 class="tb-scene__subhead">{{ labels.results.topics }}</h3>
{{/if}}
```

Значения автора хранятся у ТЕСТА и только как отклонения от ваших умолчаний, поэтому правка текста в новой версии шаблона доезжает до всех тестов, где автор эту надпись не трогал. У отчёта есть свой слой переопределений: автор может сказать в документе иначе, чем на экране.

Гейт снимает ВЕСЬ элемент, который надпись подписывает, а не только её текст. Тумблер у автора называется «Показывать», и он ожидает, что выключенная строка исчезнет целиком — вместе со значением, цветом и значком. Плата за половинчатый гейт видна сразу: пилюля вердикта темы

без слова остаётся цветным прямоугольником и продолжает сообщать исход, который автор скрыл, а строка «Правильно» без подписи оставляет на карточке осиротевшее «4 / 5». Поэтому надпись, которая подписывает слот, гейтит слот, а не себя:

```
{{#if @root.labels.topic.correct}}{{#if total}}
<div class="ou-stat-row">
  <div class="ou-stat-row__title">{{ @root.labels.topic.correct }}</div>
  <span class="tb-stat-val">{{correct}} / {{total}}</span>
</div>
{{/if}}{{/if}}
```

Гейта два, и оба нужны: данных может не быть (тема без оценивания), а надпись может быть выключена — это разные причины не печатать строку.

3.5 Состав и порядок подблоков итогов: `resultsBlockOrder`

Экран итогов состоит из пяти подблоков — `summary` (сводка баллов), `scales`, `indicators`, `topics` и `breakdown` (сводный разрез по тесту, §7.3). Раздел объявляет, какие из них печатает ВАШ шаблон и в каком порядке, отдельно по экранам:

```
"resultsBlockOrder": {
  "default": ["summary", "scales", "indicators", "topics", "breakdown"],
  "results.adaptive": ["topics", "scales", "indicators", "breakdown"]
}
```

- Форма та же, что у надписей: `default` отвечает за экраны без отдельной записи, именованный ключ экрана переопределяет список ЦЕЛИКОМ. Плоский массив вместо объекта тоже допустим — это «один список на все экраны».
- Список несёт сразу два решения: состав (ключа нет в списке — на этом экране такого подблока не бывает; адаптивные итоги никогда не печатали сводку баллов) и порядок (позиция в массиве).
- Раздела нет — действует зашитое умолчание `summary`, `scales`, `indicators`, `topics`, `breakdown`.
- Незнакомый ключ отбрасывается молча: манифест — данные, и рендер из-за него падать не должен.

Автор теста может переставить подблоки под себя; его порядок фильтруется вашим списком (чего шаблон не печатает, того автор не включит), а подблок, который вы добавили позже, дописывается в конец — уже собранный тест его не теряет. Готовый результат приходит макету массивом `result.blocks[]` (§7.3), и макет обходит его ОДНИМ циклом вместо пяти фиксированных секций.

4. Макеты и слоты

Макет — это фрагмент HTML (не целый документ: без `<html>` / `<head>` / `<body>`).

4.1 Оболочка (shell.html)

Единственное обязательное требование к оболочке — наличие области страницы `data-slot="page"`, в которую ядро монтирует текущий экран:

```
<div class="tb-player" data-template="my-template">
  <div class="tb-progress-wrap">
    <div class="tb-progress-bar" id="tb-progress-fill"></div>
  </div>
  <main id="app" class="tb-content" data-slot="page" tabindex="-1"></main>
</div>
```

Кнопки навигации (`data-nav="next"`, `data-action="…"`) ядро привязывает само, когда они присутствуют. Текущий валидатор и проверка работоспособности их наличие в оболочке **не требуют** (встроенный `default` объявляет их в макетах страниц, а не в `shell.html`).

По умолчанию `shell.html` — только внешний каркас, а экраны ядро монтирует в свой базовый контейнер. Если оформление шаблона привязано к структуре оболочки — фикс-сцена заданного размера или на единицах `cqh` / `cqw` (например `.tb-frame > .tb-stage > #app`), — объявите в манифесте `"mountShell": true`. Тогда ядро подставит `shell.html` целиком и отрисует каждый экран внутрь её `#app`. Без флага такой шаблон рендерится вне своей сцены, и CSS сцены не применяется — типовой симптом «предпросмотр не похож на дизайн». То же монтирование выполняет и предпросмотр, поэтому SCORM и превью выглядят одинаково. Шаблон без фикс-сцены (как встроенный `default`) флаг не объявляет.

4.2 Экран вопроса (question.html)

Обязательны два слота: текст вопроса и зона интерактива (варианты ответа). Их заполняет ядро по типу вопроса:

```
<div class="layout-question-wrap">
  <div class="q-header">
    <h1 class="q-title" data-path="course.title"></h1>
  </div>
  <div class="question-card">
    <div class="question-meta" data-path="state.questionCounterLabel"></div>
    <div class="question-text" data-slot="question-text"></div>
    <div data-slot="question-media"></div>
    <div data-slot="question-interaction"></div>
    <div data-slot="question-feedback"></div>
  </div>
</div>
```

Ядро заполняет ровно четыре слота вопроса. Основные: `question-text`, `question-interaction`. Дополнительные управляемые: `question-media` (медиа вопроса), `question-feedback` (обратная связь). Больше управляемых слотов у экрана вопроса нет.

Отсутствие `question-text` или `question-interaction` активацию не блокирует, но экран вопроса тогда отрисуется из стандартного шаблона с предупреждением, и ваше оформление вопроса применено не будет.

Подсказку и счётчик отдельными слотами объявлять не нужно — ядро их не заполняет: подсказка приходит уже внутри `question-interaction` (в разметке интерактива, элемент с классом `question-hint`), а счётчик выводится через `data-path` (`state.questionCounterLabel`). Если объявить `data-slot="question-hint"` или `data-slot="question-counter"`, слот останется пустым.

4.2.1 Вопрос-сценарий и карточка сценария в хабе: шаблону ничего добавлять не нужно

Тип вопроса «Сценарий» (задание в окне-симуляторе информационной системы) шаблон не затрагивает. Участник проходит такой вопрос так же, как любой другой: макет `question.html` тот же, а в слот `question-interaction` ядро кладёт обложку сценария (размытый первый экран и кнопку «Пройти», либо итог прежнего прогона: «Задание выполнено», «Выход досрочно: задание не выполнено», «Время вышло: задание не выполнено»). Окно правил, открывающееся по «Пройти», и сам плеер на весь экран ядро тоже рисует само.

Обложка и окно правил сверстаны ядром встроенными стилями (`style="…"`) и значениями токенов дизайн-системы, поэтому выглядят одинаково под любым шаблоном, в том числе из отдельного репозитория. Ни классов, ни переменных, ни слотов для них в шаблон добавлять не требуется, а манифест, макеты и `theme.css` остаются прежними. Шаблон, о сценариях не знающий, не рисует битую обложку. Так же устроена карточка пункта-сценария в хабе-маршрутизаторе: это обычная `router-topic-card` с дополнительным модификатором `router-topic-card--scenario`; стилей под модификатор в поставляемых шаблонах нет, и карточка выглядит как карточка темы.

4.2.2 Картинки на карточках маршрутизатора

Карточки хаба строит ядро, а картинка карточки — свойство оформления страницы-маршрутизатора. Чтобы карточки могли нести картинки, объявите ВТОРОЙ вариант `router` с настройкой `cardImages` типа `itemImages` и оставьте прежний вариант без неё. Автор выберет нужный командой «Сменить макет» в строке маршрутизатора, а в раскрытии строки появится список пунктов хаба с загрузкой картинки у каждого:

```
{
  "key": "router.menu-images",
  "label": "Меню карточек с картинками",
  "kind": "router",
  "pageKind": "content.router",
  "isDefault": false,
  "placeholders": [],
  "settings": [
    {
      "key": "cardImages",
      "type": "itemImages",
      "label": "Картинки карточек",
      "description": "Квадратная иллюстрация на прозрачном фоне. PNG, SVG или WebP; до 512
КБ."
    }
  ]
}
```

Подсказку `description` автор видит под списком: опишите в ней, какую картинку ждёт ваша раскладка. Карточка с картинкой получает модификатор `router-topic-card--with-img` и дочерний элемент `router-topic-card__img` с `<img>` сразу после `router-topic-card__top`; карточка без картинки не получает ни того, ни другого. Положение и размер картинки задаёт `theme.css`. Поставляемые шаблоны раскладывают такую карточку сеткой: название и метки слева, картинка справа, строка статуса на всю ширину:

```
.router-topic-card--with-img {
  display: grid; grid-template-columns: minmax(0, 1fr) auto;
  grid-template-areas: "top img" "meta img" "foot foot";
  align-items: start; column-gap: var(--ou-space-4);
}
.router-topic-card--with-img .router-topic-card__top { grid-area: top; }
.router-topic-card--with-img .router-topic-card__meta { grid-area: meta; }
.router-topic-card--with-img .router-topic-card__foot { grid-area: foot; }
.router-topic-card__img { grid-area: img; }
.router-topic-card__img img { display: block; width: 64px; height: 64px; object-fit:
contain; }
```

Чтобы предпросмотр шаблона показал раскладку на демо-данных, задайте темам в `demo/course.json` поле `image` (адрес или `data:`-URI): демо-картинки показывает только вариант, объявивший `cardImages`.

Для разработчика ядра: разметку обложки и окна правил отдают экспорты глобала `TBTemplate` в пакете (`shared/template/runtime-entry.ts`): `renderSimCover`, `renderSimRulesDialog`, `simCoverState`, `simCoverShot`, `simRules`, `simRunReplaces`, а также `mountPlayer` и `simulationRatio` (плеер и оценка прогона), `encodeSimProtocol` и `simProtocolChunks` (протокол в отчёт LMS). Это внутренний слой ядра, а не часть формата: из `template.js` на них опираться нельзя, а потому они версию контракта не двигают.

4.3 Контентная страница (content.html)

Контентная страница (introduction / info / summary / router) рендерится в слот `page-content`. Ядро вставляет туда области `data-placeholder` по описанию выбранного `contentTemplate` и заполняет их значениями автора:

```
<div class="layout-content-wrap">
  <div data-slot="page-content"></div>
  <div class="navigation">
    <button type="button" class="btn" data-nav="next">Далее</button>
  </div>
</div>
```

Слот `page-content` не обязателен формально, но его отсутствие даёт предупреждение, и страница отрисуется из стандартного шаблона — объявляйте его, иначе потеряете своё оформление.

4.3.1 Как экран находит свой макет

Резолвинг макета делится на ТРИ класса — по типу экрана, а не по одному правилу.

1. Системные экраны — по фиксированному ключу `layouts[]`. Экраны `start`, `results`, `question`, `review`, `section-results`, «Введение раздела» и `system.blocked` ядро ищет по жёсткому ключу: `layouts["start"]`, `layouts["results"]`, `layouts["question"]`, `layouts["review"]`, `layouts["section-results"]`, `layouts["section-intro"]`, `layouts["system.blocked"]`. Вариант в `contentTemplates[]` для системного экрана НЕ задаёт макет — он нужен лишь для привязки страницы и для определения fallback (объявлен ли `kind`). Если ключа нет — экран берётся из стандартного шаблона.

Важная тонкость про «Введение раздела»: ядро читает ключ `layouts["section-intro"]`, а НЕ `layoutFile` варианта `intro`. Если шаблон не объявил `section-intro`, экран падает в общий контентный рендер.

2. Прочие контентные страницы — по `layoutFile` варианта. Для `info` / `summary` / `router` / `html` макет берётся из поля `layoutFile` варианта, а при его отсутствии — из общего `layouts["content"]`. Слайд галереи — частный случай `info`: это обычный вариант со своим `layoutFile`, отдельного вида страницы и отдельного правила резолвинга для него нет.

```
{
  "key": "info.text",
  "kind": "info",
  "pageKind": "content.info",
  "layoutFile": "layouts/content-info.html",
  "placeholders": [{ "key": "body", "type": "richText", "label": "Текст" }]
}
```

Ключа вида `layouts["content.info"]` в контракте нет: такой ключ ядро не читает, макет окажется недостижим, и страница отрисуется общим `layouts["content"]`.

Placeholders варианта должны соответствовать макету: ядро вставляет области `data-placeholder` по описанию варианта. Если вариант объявляет `title` / `subtitle`, а макет ожидает `body`, значения автора отрисованы не будут.

4.4 Стартовая страница и результаты

Эти макеты целиком на DSL и `data-path` (управляемых слотов нет). Фрагмент старта:

```
<div class="start-page">
  <h1 class="start-title" data-path="course.title">Тест</h1>
  <p class="start-description" data-path="course.description"></p>

  <div class="start-info-grid">
    <div class="info-row">
      <div class="info-row-label">Количество вопросов</div>
      <div class="info-row-value" data-path="course.questionCount"></div>
    </div>
    {{#if course.timeLimitLabel}}
    <div class="info-row">
      <div class="info-row-label">Ограничение времени</div>
      <div class="info-row-value" data-path="course.timeLimitLabel"></div>
    </div>
    {{/if}}
  </div>

  <div class="start-actions">
    {{#if state.canResume}}<button class="btn" data-action="resume">{{state.resumeLabel}}
</button>{{/if}}
    {{#if state.canStart}}<button class="btn" data-action="start-test">{{state.startLabel}}
</button>{{/if}}
  </div>
</div>
```

Варианты стартового экрана. Стартовый экран поддерживает несколько раскладок: объявите дополнительные `contentTemplates[]` с `kind: "start"`, у каждой — свой `layoutFile` (напр. `start.image-right` с колонкой-иллюстрацией справа). Пометьте базовую вариантом `isDefault: true`. Автор выбирает раскладку во вкладке «Структура» → «Сменить вариант» на строке «Старт», а рантайм (и SCORM-пакет, и веб-хост) рендерит выбранный вариант по его `layoutFile` — как для контентных страниц. Если вариант не выбран или его макет не поставлен, экран падает на `layouts["start"]`. Иллюстрацию задаёт СВОЙСТВО СТРАНИЦЫ — единственное место, где её выбирают: вариант с колонкой-иллюстрацией объявляет `settings: [{ "key": "image", "type": "image", "label": "Изображение" }]`, и автор загружает картинку там же, где выбрал вариант, — в «Структуре», раскрыв строку «Старт». Параметра оформления для неё больше нет: одна настройка в двух местах приводила к тому, что автор искал её не там, а стандартный вариант старта рисовал ту же колонку и «Сменить вариант» ничего видимо не менял.

Вариант, который свойства НЕ объявляет, иллюстрацию не показывает. Значение приходит в макет биндингом `design.startImageUrl`; правило разрешения — `shared/template/start-image.ts` (`startImageForVariant`), общее для обоих плееров и предпросмотров редактора.

Значение старого параметра оформления `startImageUrl`, если оно осталось в настройках теста, ещё читается как запасное — но задать его в редакторе уже нельзя.

---

5. DSL рендерера

Движок понимает подмножество синтаксиса в стиле mustache. Поддерживается:

Конструкция	Назначение
`{{ path }}`	Экранированная подстановка текста
`{{#if path}} … {{/if}}`	Блок при истинном `path` (пустой массив = ложь)
`{{#unless path}} … {{/unless}}`	Блок при ложном `path`
`{{#each path}} … {{/each}}`	Перебор массива
`{{> name}}`	Подключение частичного шаблона (partial)

Внутри `{{#each}}` доступны: текущий элемент как контекст (его поля — `{{fieldName}}`; сам элемент-примитив — `{{ this }}` или `{{ . }}`), `{{@index}}`, `{{@number}}` (индекс + 1), `{{@first}}`, `{{@last}}`, а также `{{@root}}` — корневой контекст, от которого можно идти вглубь по пути (`{{ @root.course.title }}`).

```
{{#each result.topicResults}}
<div class="topic-card">
  <div class="topic-name">{{@number}}. {{topicName}}</div>
  <div class="results-pill {{passClass}}">{{statusLabel}}</div>
  <div class="val">{{correct}} / {{total}}</div>
</div>
{{/each}}
```

Авторский текст с форматированием — `{{& путь }}`:

Обратную связь теста и толкования уровней автор пишет в редакторе, где выбирает формат: «Обычный», «Форматированный» или «HTML». Готовую разметку ядро кладёт в ПАРНОЕ поле контекста — `textHtml` рядом с `text`, `textsHtml` рядом с `texts`, — а печатает её `{{& путь }}`: единственная запись, которая выводит значение разметкой.

```
{{#each result.recommendations.textsHtml}}<p class="rec">{{& this }}</p>{{/each}}

{{#if text}}<div class="level-text">{{& textHtml }}</div>{{/if}}
```

Гейт ставится на строку (`{{#if text}}`), а печатается разметка. Обычный текст ядро экранирует само и переводит переводы строк в `<br>`, поэтому абзацы автора доходят до слушателя. Печатать `{{& ... }}` над полем, которое ядро не готовило как разметку, нельзя — для всего остального `{{ ... }}`.

Шаблон, который печатает только `{{ text }}`, остаётся рабочим: слушатель увидит текст, но без форматирования.

Что НЕ поддерживается (по дизайну):

- JavaScript, хелперы и выражения внутри `{{ ... }}` (только путь, без пробелов).
- Вывод через `{{{ ... }}}` — выбрасывает ошибку: у контролируемого HTML одна запись, `{{& ... }}`.
- Любой `{{ ... }}` экранируется как текст. Богатый HTML СТРАНИЦЫ вставляется через управляемые `data-slot` / `data-placeholder`, не через DSL.

Невалидный шаблон (незакрытый блок, `{{{ }}}`, выражение с пробелом) отклоняется блокирующей структурной ошибкой `LAYOUT_TEMPLATE_SYNTAX` уже при загрузке пакета: сервер компилирует каждый макет (layout / partial / системную страницу / `contentTemplates[].layoutFile`), и невалидный DSL не даёт активировать шаблон. Это происходит до проверки работоспособности, а не в ней.

---

6. Привязка данных: четыре механизма

Механизм	Где	Что делает	Источник
`{{ path }}`	В HTML макета	Экранированный текст + управление потоком	Контекст рендера
`data-path="x.y"`	Атрибут элемента	`textContent` элемента из пути (экранированно)	Контекст рендера
`data-slot="name"`	Атрибут контейнера	`innerHTML` управляемого HTML	Ядро (вопросы, контент)
`data-placeholder="key"`	Атрибут контейнера	Значение поля контентной страницы по типу	Значения автора

`{{ path }}` и `data-path` читают одно и то же — выбирайте по удобству: `data-path` удобен, когда нужно оставить «заглушку» в статической вёрстке; `{{ }}` — когда значение встроено в текст или внутри условия/цикла.

7. Публичный контекст рендера

Макеты читают только публичный контекст (`shared/template/context.ts`). Каждое пространство имён присутствует лишь на тех экранах, где применимо: `course` , `state` , `result` , `retake` , `transition` , `design` , `review` , `sectionResult` , `sectionIntro` , `page` и `labels` (последнее — только на экранах итогов, адаптивных итогов, итогов раздела и в отчёте).

7.1 `course.*` (старт, вопрос, результаты)

Поле	Тип	Описание
`title`	string	Название теста
`description`	string	Описание
`questionCount`	number	Число вопросов
`passPercent`	number|null	Проходной балл, %. `null` у измерительной методики — теста, где ни один вопрос не проверяется (распределение баллов, шкала без верной градации). Порог у такого теста задан всегда: он ставится по умолчанию при создании, и автор опросника его не открывает, поэтому обложке его не показывают. Гасите факт через `{{#if course.passPercent}}` — обе поставляемые раскладки так и делают
`timeLimitMinutes`	number|null	Лимит времени числом минут, как его задал автор
`timeLimitLabel`	string	Он же готовой надписью: «14 дней», «1 день 2 ч», «2 ч 30 мин», «45 мин». Печатайте её и на неё же ставьте гейт: движок не умеет считать, и макет с зашитым «мин» на двухнедельном лимите скажет «20160 мин». Пустая строка = лимита нет
`maxAttempts`	number|null	Разрешено попыток
`startPageContent`	string	Легаси-текст введения (перенесён в контентную страницу; обычно пустой)
`closesOnLeave`	boolean	Только на старте, с версии 3.13.0. `true`, когда выход из начатого раздела закроет его навсегда (в тесте без разделов — завершит попытку): автор включил «Закрывать раздел при выходе», и у теста есть лимит времени. Чтобы предупредить до начала — `{{#if course.closesOnLeave}}`. У теста без настройки поля нет; поставляемые шаблоны его не печатают, ядро при входе в раздел тоже молчит — правило пишет автор на вводной странице
`passCondition`	object	Только на старте, с версии 3.14.0. Тематическая часть условия прохождения, когда исход решают темы: `countLabel` («8 из 8») и `label` («обязательных тем для прохождения» / «тем для прохождения»), а также `count`, `total`, `kind` (`required` / `all`). Поля нет при политике «Только общий результат», у измерительной методики и у теста без тем с порогом. При политиках «Все обязательные темы» и «Все темы» `passPercent` пуст — общий процент там справочный. Поставляемые шаблоны печатают плитку `tb-facts__item` под гейтом `{{#if course.passCondition}}`

7.2 `state.*` (старт, вопрос)

Поле	Описание
`questionCounterLabel`	Подпись счётчика, напр. «Вопрос 1 из 10»
`canStart` / `startLabel`	Показать кнопку старта и её подпись
`canResume` / `resumeLabel` / `resumeNote`	Возобновление попытки
`canRestart` / `canViewResults`	Перезапуск / просмотр сохранённого результата
`exhausted`	Попытки закончились
`showBack`	Веб-действие «назад к списку» (SCORM опускает)
`questionsProgress`	Кликабельные пилюли прогресса текущего охвата (см. ниже)
`cooldown`	Кулдаун на старте: `{ availableDateHuman, daysUntil? }`; при наличии кнопка старта отключена и показана карточка кулдауна (FR-20)
`priorResult`	Сводка прошлой попытки: `{ percent, verdictLabel, verdictClass, attemptsLabel }` (FR-20)
`canDownloadReport`	Показать «Скачать отчёт» по прошлой попытке (FR-20)

`state.questionsProgress` — карта пилюль текущего охвата навигации: `scopeLabel`, `total`, `answeredCount`, `skippedCount` и массив `states[]`. Каждая пилюля: `index` (абсолютный 0-based индекс — цель перехода `goto:<index>`), `number` (1-based в охвате), `statusClass` (`is-answered` / `is-current` / `is-skipped` /…), `ariaLabel`, `clickable` (достижима ли для перехода).

7.3 `result.*` (результаты)

Поле	Описание
`passed`	Тест пройден (boolean)
`passClass`	Готовый класс `is-pass` / `is-fail`
`statusLabel`	Готовая подпись статуса
`scorePercent`	Процент результата
`ringDashoffset`	Готовое смещение для SVG-кольца
`totalQuestions` / `correct` / `earnedPoints` / `possiblePoints`	Сводка баллов
`topicResults[]`	Результаты по темам (`topicId` , `topicName` , `correct` , `total` , `percent` , `passClass` , `statusLabel` , `recommendedCourses[]` , `recommendedEvents[]` , `hasRecommendations` ; SCORM-доп.: `pointsLabel` , `requiredLabel` ; необязательные `breakdown[]` , `interpretationHtml` , `hasAside` , `hasBreakdownNotes` — см. ниже). Текста обратной связи темы в строке НЕТ — он печатается консолидированным блоком `result.recommendations`
`topicGroups[]`	Карточки тем, разложенные по НАЗВАННЫМ блокам автора, со счётчиком «пройдено N из M» (см. ниже). Поля нет, если блоков нет
`ungroupedTopics[]`	Карточки, не попавшие ни в один блок, — печатаются после всех блоков. Поля нет, если блок нашёлся каждой
`breakdown[]`	Сводный разрез по ВСЕМУ тесту: одна строка на ключ, отдельный подблок итогов (см. ниже). Поля нет, если автор блок не выбрал
`blocks[]`	Видимые подблоки итогов, уже отобранные и упорядоченные ядром (§3.5) — по ним макет строит тело экрана
`introHtml`	Вводный блок: авторский текст, который идёт ПЕРВЫМ, до сводки, тем и измерений. Приходит разметкой — печатается через `{{& … }}` ; пусто или поля нет = блока нет
`adaptive`	Признак адаптивного режима; при `true` строки `topicResults[]` имеют форму уровней (`levelLabel` , `levelClass` , `feedback` , `hasFeedback` , `hasLinks` , `links[]`) вместо баллов
`recommendedCourses[]` / `recommendedEvents[]`	Рекомендации по проваленным темам (SCORM; веб опускает)
`backAction` / `backLabel`	Действие и подпись «назад» — устаревшая однокнопочная форма подвала; используется только когда `result.nav` не заполнен

Поле	Описание
`nav`	Состояние подвала экрана результатов: `showReport`, `canRetry`, `primaryAction`, `primaryLabel` (см. ниже)
`canRetry` / `showFinish` / `hasScormActions`	Флаги действий адаптивного экрана результатов (`results.adaptive`, SCORM)
`showPdf`	УСТАРЕЛО. Прежний флаг кнопки отчёта в `results.adaptive`; поставляемые макеты читают `result.nav.showReport`. Оставлен только для внешних шаблонов, написанных до унификации

Помимо перечисленного, `result.*` несёт кастомные переменные результата (ядро добавляет их по ключу — их можно подставлять по имени пути).

`result.nav` — состояние подвала экрана результатов (`results` и `results.adaptive`). Состав кнопок принадлежит ШАБЛОНУ, хост лишь заполняет состояние и привязывает действия по `data-action`. Макет обязан предусмотреть кнопки для всех значений, иначе соответствующая возможность у слушателя пропадёт:

Поле	`data-action`	Когда показывать
`showReport`	`download-report`	Отчёт по попытке доступен. В SCORM-пакете это «Скачать отчёт» (PDF собирается внутри пакета). Флаг приходит и на итоговом экране, и на экране «Мой результат»
`canRetry`	`restart`	Тест не пройден и попытки остались
`primaryAction` / `primaryLabel`	`results-next` либо `results-finish`	Замыкающее действие: `results-next` («Далее»), если после теста есть контентные страницы, иначе `results-finish` с подписью из `primaryLabel`

Стартовый макет несёт свою кнопку отчёта по прошлой попытке — `state.canDownloadReport` с тем же `data-action="download-report"` (FR-20).

Важно: классы и подписи (`passClass`, `statusLabel`, `ringDashoffset`) уже вычислены ядром. DSL не считает их сам — просто подставляйте готовые значения.

Надписи и подблоки — `labels.*` и `result.blocks[]`. Если шаблон объявил надписи (§3.4), контекст несёт дерево `labels.*` с уже РАЗРЕШЁННЫМИ текстами: умолчание шаблона, поверх него значение автора, а для отчёта — ещё и его собственный слой. Пустая строка означает «не печатать», поэтому гейт ставится на самом значении, отдельного флага видимости нет. Ключ, который шаблон не объявил, приходит пустой строкой — экран ученика из-за макета не падает.

`result.blocks[]` — состав и порядок подблоков (§3.5), уже готовые. Каждый элемент несёт `key`, эффективный `heading` (пустая строка — автор погасил заголовок, сам блок остаётся) и ОДИН булев флаг вида: `isSummary`, `isScales`, `isIndicators`, `isTopics`, `isBreakdown`. Флаги нужны потому, что DSL не умеет сравнивать строки — `{{#if key == "scales"}}` написать нечем.

```
{{#if result.introHtml}}<div class="tb-intro">{{& result.introHtml }}</div>{{/if}}
{{#if labels.results.heading}}<h2 class="tb-scene__head">{{ labels.results.heading }}</h2>
{{/if}}
{{#each result.blocks}}
  {{#if heading}}<h3 class="tb-scene__subhead">{{ heading }}</h3>{{/if}}
  {{#if isSummary}}…сводка баллов…{{/if}}
  {{#if isScales}}…шкалы…{{/if}}
  {{#if isIndicators}}…показатели…{{/if}}
  {{#if isTopics}}…карточки тем…{{/if}}
  {{#if isBreakdown}}…сводный разрез…{{/if}}
{{/each}}
```

Зонтичный заголовок печатайте только вместе с непустым `result.blocks`: видимых подблоков нет — надписывать нечего. Порядок подблоков обязан выражаться ДАННЫМИ, а не CSS: конвейер отчёта читает реальный порядок DOM, и перестановка средствами стилей его обманет.

Разрез результата — `topicResults[].breakdown[].` Карточка темы может нести ещё один уровень: подытоги по ключам (сегодня это подтемы-теги вопросов) в пределах ЭТОГО раздела. Строки приходят готовыми, считать нечего:

Поле	Описание
`key`	Ключ — подпись строки (название подтемы)
`items` / `answered`	Сколько выданных вопросов несут ключ и сколько из них отвечено
`earned` / `possible`	Баллы по ключу — заработанные и возможные, один знак после запятой
`percent`	Число по ВЫБРАННОЙ автором базе, один знак — печатайте его, если рисуете своё значение, и вам не придётся выяснять, какая база в силе
`percentUnits` / `percentPoints`	Доля вопросов и доля баллов по отдельности, по одному знаку
`barPercent`	Ширина полосы в процентах, округлена до целого
`showValue`	Печатать ли число рядом с полосой
`valueLabel`	Готовая подпись значения («50 %»); пуста при `showValue: false`
`passed`	Исход подтемы: `true` / `false` / `null` — порога у ключа не было
`passClass`	Готовый модификатор строки: `is-pass`, `is-fail` или пустая строка. Пуст и при окраске «По доле» или «Нейтральная»
`barFill`	Готовая заливка полосы (`linear-gradient( … )`); приходит только при окраске «По доле» и непустой полосе
`requiredLabel`	Готовая надпись порога («Нужно 70 %»); поля НЕТ, когда порога нет или показ значения выключен

Числа лежат в строке всегда, даже когда поставляемые шаблоны рисуют одну полосу: без них вы не напечатали бы «верно 4 из 7», а появись они позже — это была бы правка контракта, которую пришлось бы возить по обоим хостам и по всем собранным пакетам.

Исход в строке есть, слова — нет. С версии 3.1.0 строка снова несёт `passed` и `passClass`, а рядом с ними — `requiredLabel`. Версия 3.0.0 снимала весь набор; вернулась только та его часть, что говорит ТОНОМ. `statusLabel` не вернулся: строка узкая, и слово «Не пройдено» рядом с процентом дублировало бы цвет, а вердикт СЛОВОМ выносит тема, чья карточка стоит вокруг строк.

Красьте ровно одну вещь — полосу. Порога у ключа нет — `passed` равен `null`, `passClass` пуст, `requiredLabel` не приходит, и строка выглядит ровно так, как выглядела до 3.1.0. Надпись порога приезжает только там, где автор включил показ ЗНАЧЕНИЯ: цвет без числа и без причины читается как приговор.

Что кодирует цвет полосы, решает автор теста (с 3.12.0). Объявите в манифесте параметр `breakdownBarFill` (тип `select`, группа «Итоги») со значениями `share`, `verdict` и `neutral`, и в редакторе появится пункт «Окраска полос подтем». `default` выберите сами: он и станет видом теста, в котором ничего не настраивали.

- `verdict` — как раньше: строка несёт `passClass`, цвет задаёт ваш CSS.
- `share` — `passClass` пуст, а `barFill` несёт готовый градиент в цветах схемы уровней теста: прямой переход от неблагоприятного края до цвета в точке доли, без жёлтого и зелёного посередине, которые дала бы дуга оттенка. Считать ничего не нужно, как и знать цвета.
- `neutral` — ни класса, ни заливки: полоса вашего нейтрального цвета.

Печатайте `barFill` в `style` полосы, ПОСЛЕ ширины, как в примере ниже. Инлайн-значение перебивает цвет заливки из вашего CSS, поэтому отдельных правил под режимы писать не нужно. Если параметр не объявлен, строка ведёт себя как при `verdict`.

```
{{#if breakdown}}
<div class="tb-topic__breakdown">
  {{#each breakdown}}
  <div class="tb-breakdown__row {{ passClass }}" data-item="{{ key }}">
    <div class="tb-breakdown__name">{{ key }}</div>
    <div class="tb-breakdown__bar"><span style="width: {{ barPercent }}%;{{#if barFill}}
background: {{ barFill }};{{/if}}"></span></div>
    {{#if showValue}}<div class="tb-breakdown__val">{{ valueLabel }}</div>{{/if}}
    {{#if requiredLabel}}<div class="tb-breakdown__req">{{ requiredLabel }}</div>{{/if}}
  </div>
  {{/each}}
</div>
{{/if}}
```

Что об этом полезно знать верстальщику:

- поля НЕТ вовсе, когда автор теста показ не включил или в теме нет ни одного ключа: один гейт `{{#if breakdown}}` убирает блок вместе с заголовком;
- показ включает автор теста настройкой «Подытоги по ключам» (не показывать / полоса / полоса и число) и там же выбирает «Базу подытогов» — долю вопросов или долю баллов. Настройка одна на экран итогов и на отчёт;
- это СПИСОК КЛЮЧЕЙ, а не разложение темы на части: ключи не обязаны покрывать все вопросы, и сумма строк не обязана сходиться с итогом темы. Не подписывайте блок как «состав раздела»;
- вердикт от базы показа не зависит — пороги всегда считаются в баллах, полоса лишь показывает.

Блоки разделов — `topicGroups[]` и `ungroupedTopics[]`. Автор теста может собрать разделы в названные блоки («Обязательная часть», «Дополнительная часть»), и тогда карточки тем печатаются по блокам, каждый со своим счётчиком. Вложенность ровно одна — тест, блок, разделы:

Поле блока	Описание
`key` / `label`	Ключ блока и его заголовок; заголовок может быть пустым
`topics[]`	Карточки блока в порядке выдачи — те же объекты, что и в `topicResults[]`
`passedCount` / `totalCount`	Разделы с вердиктом «пройден» и разделы с любым вынесенным вердиктом
`counterLabel`	Готовый счётчик («1 / 2»)

Ловушка одна, и она обязательна к пониманию: `topicResults[]` рядом с блоками остаётся ПОЛНЫМ — там и сгруппированные карточки, и остальные. Печатать его РЯДОМ с блоками нельзя, тема выйдет дважды. Поэтому макет гейтится на `topicGroups` и печатает либо блоки плюс `ungroupedTopics`, либо плоский список:

```
{{#each result.topicGroups}}
  <div class="tb-topic-group">
    {{#if label}}<h4>{{ label }}</h4>{{/if}}
    {{#if counterLabel}}<span>{{ counterLabel }}</span>{{/if}}
    {{#each topics}}…карточка темы…{{/each}}
  </div>
{{/each}}
{{#if result.ungroupedTopics}}{{#each result.ungroupedTopics}}…карточка темы…{{/each}}
{{/if}}
{{#unless result.topicGroups}}{{#each result.topicResults}}…карточка темы…{{/each}}
{{/unless}}
```

Отдельный список несгруппированных нужен потому, что DSL не умеет фильтровать: без него макет не отличил бы уже напечатанные карточки от оставшихся. Шаблон, ничего не знающий о блоках, печатает плоский список и работает как прежде.

Сводный разрез по тесту — `result.breakdown[]`. Разрез в карточке темы живёт в пределах раздела: один и тот же ключ в двух разделах даёт две строки, и складывать их макет не вправе. Сводный блок отвечает на другой вопрос — «как ключ выглядит по всему тесту» — и приходит ОТДЕЛЬНЫМ подблоком итогов (`isBreakdown`), одной строкой на ключ. Строки того же типа, что и в карточке, поэтому вёрстка повторяется:

```
{{#if isBreakdown}}
<div class="tb-breakdown">
  {{#each @root.result.breakdown}}
  <div class="tb-breakdown__row" data-item="{{ key }}">
    <div class="tb-breakdown__name">{{ key }}</div>
    <div class="tb-breakdown__bar"><span style="width: {{ barPercent }}%;{{#if barFill}}
background: {{ barFill }};{{/if}}"></span></div>
    {{#if showValue}}<div class="tb-breakdown__val">{{ valueLabel }}</div>{{/if}}
  </div>
  {{/each}}
</div>
{{/if}}
```

Внутри `{{#each result.blocks}}` до самих строк добираются через `@root` — контекст цикла указывает на подблок, а не на результат. Что ещё полезно знать:

- где печатать разрез, выбирает автор теста третьим полем настройки: в карточках тем, сводным блоком или и там, и там. Настройка, сохранённая до появления блока, читается как «в карточках»;
- заголовок блока — надпись `results.breakdown` («Разрез результата» по умолчанию), позиция — `resultsBlockOrder` (§3.5), в поставляемом порядке он последний;
- на АДАПТИВНЫХ итогах работает только сводный блок: карточка темы там говорит подтверждённым уровнем и полос не несёт. Объявляйте `breakdown` и в списке `results.adaptive`;
- в отчёт приходит то же самое: `result.*` документа строит тот же построитель.

Толкование темы и подтемы — `interpretationHtml`. Это текст, который ОБЪЯСНЯЕТ результат: что проверяет тема и что стоит за полученной оценкой. С обратной связью его легко спутать, а правила выдачи у них разные, и разница — не тон, а вот что:

	Толкование	Обратная связь
Когда	ВСЕГДА, при любом вердикте	тема не взята; доля подтемы ниже общего порога теста
Где	ПРИ своей теме или полосе	в сводном блоке `result.recommendations`
Что несёт	только текст	текст, курсы, материалы, мероприятия

Печатать надо через `{{& … }}`, а не через `data-path`: ядро отдаёт готовую разметку, а проход по `[data-path]` заполняет узел через `textContent` и затёр бы абзацы.

```
<div class="tb-topic-card{{#if hasAside}} tb-topic-card--split{{/if}}">
  <div class="tb-topic-card__main">
    <span class="tb-topic-card__name">{{topicName}}</span>
    {{#if statusLabel}}<span class="ou-tag {{passClass}}">{{statusLabel}}</span>{{/if}}
  </div>
  <div class="tb-topic-card__aside">
    {{#if interpretationHtml}}<div class="tb-topic-card__interpretation">{{&
interpretationHtml }}</div>{{/if}}
    {{#if breakdown}}
    <div class="tb-topic__breakdown{{#if hasBreakdownNotes}} tb-topic__breakdown--
notes{{/if}}">
      {{#each breakdown}}
      <div class="tb-breakdown__row {{ passClass }}" data-item="{{ key }}">
        <div class="tb-breakdown__name">{{ key }}</div>
        <div class="tb-breakdown__bar"><span style="width: {{ barPercent }}%;{{#if
barFill}} background: {{ barFill }};{{/if}}"></span></div>
        {{#if interpretationHtml}}<div class="tb-breakdown__note">{{& interpretationHtml }}
</div>{{/if}}
      </div>
      {{/each}}
    </div>
    {{/if}}
  </div>
</div>
```

Что ещё полезно знать:

- поля НЕТ вовсе, когда текста нет: гейт стоит на самом поле, пустая строка печатала бы пустой блок;
- текст ТЕСТА заменяет текст ТЕМЫ целиком — разрешает ядро, макет получает готовое значение;
- `hasAside` и `hasBreakdownNotes` считает ядро: движок не умеет ни перебрать строки, ни сложить два вопроса в один ответ. Первый говорит «у темы есть что показать второй колонкой», второй — «хоть одна полоса несёт свой текст», и оба нужны РАСКЛАДКЕ, а не данным;
- толкования подтем приезжают только при включённой настройке теста «Показывать толкование подтем»; выключенная не стирает текстов, они просто не попадают в контекст;
- в блок `result.recommendations` толкование не попадает никогда — иначе одна мысль напечаталась бы дважды.

Заголовки итога. Тест вправе назвать свой итог сам, и для этого вариант `results.*` объявляет три настройки (`settings[]`):

Ключ	На что действует
`headingDocument`	`course.title` — и на экране, и в отчёте
`headingPassed`	`result.statusLabel` на экране, `report.verdictHeadline` в отчёте
`headingFailed`	там же при неуспехе

Новых полей контекста тут нет: значения приходят в прежние, и макет правок не требует. Но объявить ключи обязан шаблон — иначе автор просто не увидит полей. Пустое значение означает умолчание, и умолчания у экрана и у документа РАЗНЫЕ («Пройден» против «Тест пройден»): над шапкой документа нет названия теста, и одним словом она не держится.

7.3а Отчёт о результатах: `report.*`

Отчёт — это PDF, который обучающийся скачивает с экрана результатов. Он НЕ снимок экрана: у него своя страница со своим макетом, который объявляет шаблон. Видов два, и они **разные**, а не варианты одного:

`kind`	Когда применяется	Что печатает
`report`	обычный тест	баллы, проценты, вердикт
`report.adaptive`	адаптивный тест	подтверждённые уровни, без баллов

Объявление варианта — обычная запись `contentTemplates[]`. Обязателен `layoutFile`: общего макета «на весь вид» у отчёта нет. Необязателен `styleFile` — таблица стилей страницы. Ровно один вариант каждого вида помечается `isDefault`.

```
{
  "key": "report.certificate",
  "label": "Сертификат: с подложкой",
  "kind": "report",
  "layoutFile": "layouts/report.html",
  "styleFile": "styles/report.css",
  "isDefault": true,
  "settings": [
    { "key": "headline", "type": "text", "label": "Заголовок отчёта", "default": "Итоги" },
    {
      "key": "backgroundImage",
      "type": "image",
      "label": "Подложка страницы",
      "default": "assets/report/bg.png"
    },
    { "key": "showRecommendations", "type": "boolean", "label": "Показывать рекомендации",
"default": true }
  ]
}
```

`settings[]` — те же поля, что у контентных страниц (§8.1). Ограничения отчёта: тип `sequence` неприменим (последовательностей у отчёта нет), `placeholders[]` неприменимы (в отчёте нет содержимого, которое обучающийся читает как страницу).

Скажите, где автор будет искать поле, — `scope`. Настройки отчёта разложены по двум экранам редактора, и раскладывает их ваш манифест:

```
{ "key": "scalesChartKind", "type": "select", "scope": "content", "label": "Диаграмма по
шкалам" },
{ "key": "backgroundImage", "type": "image", "label": "Подложка страницы" }
```

- `scope: "content"` — поле решает, ЧТО попадёт в документ. Автор правит его в разделе «Настройки», в блоке обратной связи, рядом с текстом, который прочтёт слушатель.
- `scope: "appearance"` (умолчание) — поле решает, КАК документ выглядит. Автор правит его в разделе «Оформление», в пункте «Отчёт», рядом с шаблоном и брендингом.

Признак необязателен: поле без него считается оформительским, и шаблон, написанный до появления `scope`, продолжает работать ровно как раньше.

Картинки отчёта — ВАШИ файлы. Подложка, логотип и любая графика страницы лежат в вашем пакете, перечисляются в `assets.images` и объявляются полем типа `image`, у которого `default` — путь внутри шаблона (`assets/report/bg.png`). Ядро своей подложки и своего логотипа у отчёта

не имеет: не объявите — страница напечатается на фоне из вашего `styleFile`, и это нормальный исход, а не ошибка.

Автор теста может заменить любую из них своей картинкой; пустое поле означает «взять файл шаблона», и в интерфейсе автора умолчание показано заполнителем.

Макет читает такие поля как `report.values.<ключ>` — значение приходит ГОТОВОЙ строкой:

```
<div class="tb-report" style="{{#if report.values.backgroundImage}}background-image: url({{
report.values.backgroundImage }});{{/if}}">
  {{#if report.values.logoImage}}<img src="{{ report.values.logoImage }}" alt="">{{/if}}
```

Путь до картинки разрешает ХОСТ: внутри пакета файл лежит рядом (`template/…`), на вебе отдаётся роутом файлов шаблона. Перед созданием PDF значение инлайнится в data-URL — растеризатор снимает то, что уже в документе, и ничего не догружает. Не прочиталась — значение станет пустым, и макет просто не нарисует эту строку.

Среда стилей. Отчёт рендерится в служебном контейнере главного документа, вне сцены. Доступны:

- компоненты дизайн-системы (`.ou-*`) — как на любом экране;
- токены темы и брендинг теста: хост записывает их CSS-переменными на контейнер, поэтому работают `var(--ou-…)`, `hsl(var(--primary))`, `var(--font-sans)`;
- собственный `styleFile` варианта.

Запрещены и отклоняются при активации шаблона:

- классы слоя сцены (`tb-scene*`) — сцена описывает экран фиксированного вьюпорта, а отчёт печатается на A4. Такой макет заработал бы в SCORM-пакете и сломался в браузере;
- селекторы `:root`, `html`, `body` в `styleFile` — отчёт не документ.

Корневой элемент макета обязан нести класс `tb-report`, а все селекторы `styleFile` — быть вложены в `.tb-report`. И то, и другое проверяется статически.

Контекст страницы отчёта — это `course.*`, `design.*` и `result.*` в точности как на экране результатов (§7.3), плюс блок `report.*`:

Поле	Описание
`attemptDateLabel`	Готовая подпись даты прохождения
`attemptsCountLabel`	Готовая подпись числа попыток, со склонением; ПУСТА у теста, который ничего не оценивает (макет гейтит строку)
`learnerName` / `hasLearnerName`	ФИО слушателя и гейт строки: имя может быть неизвестно
`gridColumns`	Число колонок сетки тем, вычисленное ядром
`verdictHeadline` / `verdictBadge` / `verdictClass`	Заголовок («Тест пройден»), подпись бейджа и класс `is-pass` / `is-fail`
`correctLabel` / `earnedPointsLabel`	Готовые подписи «верно из общего числа» и заработанных баллов
`ringDasharray` / `ringDashoffset`	Геометрия дуги отчёта — своя, не как у кольца на экране итогов
`hasTopics`	Гейт блока тем
`courses[]` / `hasCourses`, `events[]` / `hasEvents`	Рекомендации: в обычном отчёте — по проваленным темам без дублей, в адаптивном — по всем темам, где они есть, с названием темы
`isPreview`	Предпросмотр в настройках теста: можно показать пометку «образец»
`values.<key>`	Значения `settings[]` варианта. Поля `image` приходят готовой строкой (путь файла шаблона уже разрешён и инлайнен в data-URL); пусто = картинки нет

Строки `result.topicResults[]` в отчёте несут дополнительные готовые подписи, которых на экране итогов нет. В обычном отчёте — `verdictLabel` («Пройден» / «Не пройден»), `barPercent`, `countsLabel` («3 из 5 (60%)») и `pointsFixedLabel` («3.0/5.0»). В адаптивном — `hasCounts`, `answeredLabel`, `correctLabel` и `achievedClass` (`is-achieved` / `is-below`), а также `feedback` / `hasFeedback` — обратная связь достигнутого уровня.

Текста обратной связи темы в строке нет ни на экране, ни в обычном отчёте: он печатается один раз, в консолидированном блоке `result.recommendations`. Строки разреза (`breakdown[]` , §7.3) приходят в отчёт в том же виде, что и на экран, — настройка показа у них одна на обе выдачи.

Надписи в отчёте. Дерево `labels.*` приходит и сюда, но со своим слоем: автор теста может сказать в документе иначе, чем на экране, а `defaults.report` в объявлении (§3.4) задаёт для отчёта другое умолчание. Важная особенность: список надписей, который редактор предлагает автору для отчёта, СЧИТАЕТСЯ по вашим макетам отчёта — сканированием на прямые пути `labels.<ключ>`. Надпись, которую макет отчёта не печатает, автору просто не покажут, и наоборот — чтобы отдать автору заголовок в документе, достаточно напечатать его в макете.

Второго списка в манифесте для этого нет: он молча разошёлся бы с макетами. Состав и порядок подблоков (`result.blocks`) отчёту не передаются — структура документа ваша, поэтому прямой путь `labels.<ключ>` здесь единственный.

Ссылка внутри отчёта становится кликабельной в PDF в двух случаях: элемент несёт класс `pdf-link-btn` и атрибут `data-url` , либо это обычная `<a href>` с адресом `http(s)://` или `mailto:` . Второе покрывает ссылки, которые автор вставляет в текст: печатайте авторский `richText` как есть, и ссылки в нём заработают без усилий с вашей стороны. Перенесённая на несколько строк ссылка нажимается на каждой строке. Якорь `#…` и относительный путь в PDF не переносятся — вести им некуда.

Принудительный разрыв страницы — `data-page-break` . Раскладка делит документ сама: по высоте листа, по границам карточек и по нижним границам строк. Она знает, что помещается на лист, но не знает, что документ читается разделами. Скажите ей это пустым узлом с атрибутом:

```
<section class="tb-report__card">…результаты по темам…</section>
<div data-page-break></div>
<section class="tb-report__card">…показатели…</section>
```

- Метка ставится на любой глубине: между карточками, внутри карточки, в теле `{{#each}}` — последнее даёт «каждое толкование показателя с новой страницы».
- Стилей она не требует и на бумаге следа не оставляет: узел пустой, а разрез идёт по его ВЕРХНЕЙ границе.
- Приказ старше всех прочих правил раскладки, включая «карточку не рвём». Метка ВНУТРИ карточки эту карточку разорвёт — с её фоном и скруглениями; это ваш выбор, продукт ему не мешает.
- Пустых листов метка не порождает: метка в самом начале документа, сразу после другого разрыва или после последнего содержимого молча игнорируется.
- Лист остаётся полноразмерным: разрыв меняет только то, где кончается содержимое, а не размер бумаги и не то, докуда доходит подложка.
- Метка внутри `{{#if}}` действует по тому же условию, что и остальная разметка, — в том числе по значению настройки варианта (`settings[]`), если решите отдать выбор автору.

Шаблон, не объявивший нужного вида, отчёта НЕ лишается: страница собирается по макету шаблона «Стандартный», а цвета и логотип берутся из параметров активного шаблона. Вместе с макетом оттуда же приезжает и его `styleFile` — иначе страница собралась бы без оформления.

Кто и что выбирает. Вариант отчёта и значения его полей — свойство ТЕСТА: автор выбирает их в блоке обратной связи раздела «Настройки», из вариантов того вида, который отвечает режиму теста (обычный тест адаптивных вариантов не видит и наоборот). Смена варианта сохраняет значения полей с совпадающими ключами и типами, остальные отбрасываются.

Предпросмотр. Оттуда же автор открывает предпросмотр: страница рисуется вашим макетом на РЕАЛЬНОЙ структуре редактируемого теста — его название и его разделы, — а баллы, проценты и вердикты подставляются демонстрационные. Переключатель показывает оба исхода, пройден и не пройден. Отсюда практическое следствие для макета: сетку тем увидят не на трёх аккуратных разделах, а на стольких, сколько их в тесте, и с настоящими длинными названиями.

Что уходит в SCORM-пакет. Пакет несёт каталог шаблона целиком, поэтому макет и ассеты варианта в нём уже есть. Сверх этого сборщик разрешает выбор автора против манифеста и запекает результат в `TEST_DATA.designSettings.report` (`layoutKey`, `styleFile`, `values`), а CSS выбранного варианта вкладывает в общий `styles.css`: к моменту растеризации читать файлы из рантайма поздно. Пакеты, собранные до появления этого контракта, запечённого выбора не несут и продолжают собирать отчёт по каноническому виду.

Проверка работоспособности (раздел «Шаблоны») рендерит по одному варианту каждого объявленного вида отчёта наравне с экранами: ошибка отрисовки макета отчёта блокирует активацию так же, как ошибка экрана. Без этого сломанный отчёт всплывал бы только у обучающегося, который не смог скачать PDF.

7.3b Документ отчёта из блоков

Отчёт собирается не одним макетом, а списком БЛОКОВ. Вы объявляете оболочку и разделы, автор теста собирает из них документ и пишет текст авторских страниц.

Шаг 1. Оболочка. Вид `report` объявляет корень документа:

```
<!— layouts/report/shell.html —>
<div class="tb-report {{ report.verdictClass }}">
{{#if report.values.logoImage}}<div class="tb-report__brand"><img src="{{
report.values.logoImage }}" alt=""></div>{{/if}}
</div>
```

Внутрь ничего не кладите: блоки движок вложит сюда сам, ПРЯМЫМИ детьми. Свой контейнер между корнем и блоками сломает разбивку на листы — она измеряет прямых детей корня.

Шаг 2. Блоки. На каждый раздел — своя запись и свой макет:

```
{
  "key": "report.block.topics",
  "label": "Темы: карточки",
  "kind": "report.block",
  "block": "topics",
  "kinds": ["report"],
  "isDefault": true,
  "layoutFile": "layouts/report/topics.html"
}
```

Ключей блоков двенадцать и перечень закрыт (спецификация §8.4.7). Вариантов одного блока может быть сколько угодно — автор выберет строкой документа.

Шаг 3. Авторская страница. Блок `page` — единственный, у которого есть `placeholders[]`: его текст пишет автор теста.

```
{
  "key": "report.block.page.text",
  "label": "Страница: заголовок и текст",
  "kind": "report.block",
  "block": "page",
  "isDefault": true,
  "layoutFile": "layouts/report/page-text.html",
  "placeholders": [
    { "key": "title", "type": "text", "label": "Заголовок" },
    { "key": "body", "type": "richText", "label": "Текст" }
  ]
}
```

```
<!— layouts/report/page-text.html —>
<section class="tb-report__card">
  <div class="tb-report__page-title" data-placeholder="title"></div>
  <div class="tb-report__text" data-placeholder="body"></div>
</section>
```

Шаг 4. Документ по умолчанию. Корневым ключом манифеста:

```
"reportDocument": {
  "report": ["header", "intro", "page-break", "topics", "summary"],
  "report.adaptive": ["header", "topics"]
}
```

`page-break` ставьте столько раз, сколько листов хотите открыть; блок с данными в списке повторять нельзя. Отдельного варианта для `page-break` объявлять не нужно — раскладки у него нет, его печатает движок.

Частые ошибки

Симптом	Причина
Документ печатается одним куском, разрывы игнорируются	Оболочка завернула блоки в свой контейнер — раскладка измеряет ПРЯМЫХ детей корня
Шаблон не проходит загрузку: «блок указан дважды»	В `reportDocument` повторён блок с данными; повторять вправе только `page-break`
Блок печатается пустым	Макет читает путь, которого в контексте нет — сверьтесь с §7.3а
Автор не видит своего текста в отчёте	`placeholders[]` объявлен у блока, отличного от `page`: там он запрещён
Отчёт печатается по-старому, целиком	Для этого вида не объявлен `reportDocument` — так и работает деградация
Надписи словаря не предлагаются автору	Надпись печатается не прямым путём `labels.<ключ>`; сканируются и оболочка, и блоки

7.4 `retake.*` (экран блокировки) и `transition.*` (адаптивный переход)

`retake`: `cooldownPeriodDays`, `availableDate`, `availableDateHuman`, `reason`. `transition`: `isCorrect`, `iconClass`, `title`, `level.{class,isUp,isDown,isComplete,message}`, `topic.toTopic`, `showContinue`.

7.5 `design.*`, `review.*`, `sectionResult.*`, `sectionIntro.*`

`design.*` (везде, где показывается брендинг): `logoUrl` — URL логотипа (разрешён в строку из media-конверта параметра), `startImageUrl` — URL иллюстрации стартового экрана (свойство страницы, §4). Блоки `{{#if design.logoUrl}}` / `{{#if design.startImageUrl}}` скрываются при отсутствии. Стартовый макет привязывает `design.startImageUrl` в правой колонке-иллюстрации: стандартный старт показывает её, когда она задана, а вариант «изображение справа» держит колонку всегда (с плейсхолдером, пока изображение не задано). `logoUrl` приходит из параметра оформления (тип `image`, секция `branding`). `startImageUrl` получает только тот вариант старта,

который объявил свойство `image` (`settings[]`, §4), и берётся он из САМОЙ страницы; варианту без свойства `startImageUrl` не передаётся вовсе. Параметра оформления с таким ключом шаблоны больше не объявляют — сохранившееся значение читается лишь как запасное.

`review.*` (экран «Обзор раздела/теста»): `scopeLabel`, `isTest` (обзор теста или раздела), `finishLabel` («Завершить …»), `unanswered[]` (`index`, `number`, `prompt`), `unansweredCount`, `answeredCount`, `total`, `hint`.

`sectionResult.*` (вычисляемый экран «Итоги раздела»): `topicName`, `scorePercent`, `ringDashoffset`, `passClass`, `statusLabel`, `hasVerdict`, `correct`, `total`, `summaryLabel` (напр. «6 из 8 верно · 75%»), `continueLabel`.

`sectionIntro.*` (экран «Введение раздела»): `eyebrow` («Раздел N»), `topicName`, `description`, `hasDescription`, `questionCount`, `questionCountLabel` (напр. «16 вопросов»), `hasTimeLimit`, `timeLimitLabel` (та же надпись, что и у лимита теста: «17 мин», «2 ч 30 мин»), `hasInstruction`, `continueLabel`. С версии 3.14.0 ещё три необязательных поля (спецификация, §10.12): `passCondition` — порог темы готовой строкой («Для прохождения: 70 %», «Для прохождения: 7 баллов из 10»; пустая строка, если тема не судится), `isRequired` — метка «Обязательная тема» там, где обязательность решает вердикт, и `timerWarning` — системное предупреждение о старте отсчёта раздела (пустая строка без лимита раздела). Поставляемые шаблоны печатают порог строкой `tb-scene__lead`, метку — тегом `ou-tag--neutral`, предупреждение — баннером `ou-banner--warning`; гейт каждого блока стоит на самом поле. С версии 3.15.0 `passCondition` пуст и тогда, когда автор выключил у страницы тумблер `passConditionShown` (см. настройки варианта введения ниже).

`page.*` (любая контентная страница): `dots[]` — точки индикатора последовательности (по одной на страницу отрезка; у текущей выставлен `statusClass`), `dotIndex` и `dotsTotal` — позиция и размер отрезка, `canGoBack` — доступен ли возврат на предыдущий экран. Всё вычисляется ядром из структуры теста, автор не вводит ничего. Страница вне последовательности получает пустой `dots[]`, поэтому макет с индикатором просто ничего не рисует — см. §8.1.

---

8. Контентные шаблоны и placeholders

`contentTemplates[]` описывает «типы» контентных страниц. Каждый элемент:

- `key` — уникальный ключ варианта (`intro.hero`);
- `kind` — функциональный вид страницы;
- `placeholders[]` — поля, которые заполняет автор.

Виды (`kind`):

`kind`	Назначение	Гранулярность
`start`	Стартовый экран теста (лендинг)	Одна на тест, всегда
`intro`	«Введение раздела» (`before_topic`)	По одной на тему (только в режимах по темам)
`info`	Учебная/информационная страница, любой контент	Сколько угодно
`questions`	Макет страницы вопроса	Одна в плоском режиме / по одной на тему
`router`	Страница-маршрутизатор (меню тем)	Одна, только `router_by_topics`
`review`	«Обзор раздела/теста» перед завершением: список неотвеченных + «Завершить»	Системный узел; своя рантайм-фаза
`section-results`	Вычисляемый «Итог раздела» после завершения (кладётся в `sectionResult.*`)	Системный узел; вне потока контентных страниц
`results`	«Итоги теста» — итоговый результат всего теста	Одна на тест, всегда
`summary`	Легаси per-topic «Итог раздела» (`after_topic`): результат раздела кладётся в `result.*` . Оставлен для обратной совместимости, роль перешла к `section-results`	Устаревший; в новых шаблонах не требуется

`start` и `results` — тест-уровневые системные экраны (лендинг и итоги теста); они присутствуют всегда и рисуются собственными рантайм-экранами (в поток контентных страниц не входят). Аналогично `review` и `section-results` — системные узлы раздела: «Обзор» перед завершением и вычисляемые «Итоги раздела» после него; они тоже рендерятся своими рантайм-фазами и **исключены** из потока контентных страниц. `intro` — «Введение раздела» перед его вопросами (по одной на тему, только в режимах по темам). `summary` — устаревший per-topic «Итог раздела»: остаётся валидным `kind` для обратной совместимости, но его роль выполняет вычисляемый `section-results` ; в новых шаблонах его объявлять не нужно.

Встроенный шаблон `default` обязан объявить хотя бы по одному варианту каждого системного `kind` : `start` , `results` , `router` , `questions` , `intro` , `review` , `section-results` (`defaultTemplateManifestSchema` , `shared/schema.ts`). Внешним шаблонам это структурно не навязывается, но при отсутствии варианта системного `kind` соответствующий экран берётся из `default` . Несколько вариантов одного `kind` (напр. два варианта `start`) — допустимы; автор выбирает нужный.

Типы placeholders и как они отрисовываются в `data-placeholder` :

`type`	Отрисовка
`text`, `textarea`	Экранированный текст, переносы строк → ` `
`richText`, `html`	HTML как есть (доверенный богатый контент)
`image`	`<img src="…">` (вписывается в контейнер)
`resultField`	Показатель результата через реестр рендереров (см. ниже)

Перечень закрыт: тип вне таблицы — ошибка валидации при загрузке и активации шаблона, с указанием ключа варианта и ключа поля. Прежние `number`, `boolean` и `select` из плейсхолдеров убраны — по природе это настройки страницы, им место в `settings[]` (ниже); `video`, `file` и `actionLabel` не поддерживаются.

Кроме `type`, у плейсхолдера могут быть дополнительные свойства, влияющие на редактор контента (не на рендер): `maxLength`, `required`, `textFit` (режим подгонки текста — `autoFitFont` / `growBox` / `fixed` плюс размеры и `overflow`). Пример — эталонный `manifest.json` (варианты `intro.standard`, `info.text`). Прежние `allowedMarks` / `allowedBlocks` из контракта исключены (см. §8.2).

Встроенные рендереры `resultField` (`shared/template/renderers.ts`): `core.textMetric`, `core.badge`, `core.progressBar`, `core.ringChart` (кольцевая диаграмма), `core.segmentedProgress`. Шаблон ограничивает выбор рендерера через `allowedRenderers`, а путь данных — через `allowedPaths` плейсхолдера. Реестр устойчив к ошибкам: рендерер не из `allowedRenderers`, неизвестный или упавший откатывается к `core.textMetric`; путь не из `allowedPaths` даёт пустое значение.

Пример `resultField` в наборе данных автора (значение `values.result`) — кольцо:

```
{
  "path": "result.scorePercent",
  "renderer": "core.ringChart",
  "rendererOptions": { "showValue": true, "decimals": 0, "size": 150, "strokeWidth": 14 },
  "label": ""
}
```

На странице `summary` («Итог раздела») `result.scorePercent` — это результат **раздела** (Core подаёт его в `result.*`, §8.2 платформенной спеки).

8.1 Свойства страницы: `settings[]`

`placeholders[]` — это СОДЕРЖИМОЕ, которое вставляется в макет. Всё, что управляет поведением или оформлением самой страницы, объявляется отдельно, в `settings[]`. Раздел необязателен: без него у страниц варианта настраиваемых свойств нет, и форма выглядит как сегодня.

```
{
  "key": "gallery.card",
  "label": "Галерея: карточка",
  "kind": "info",
  "layoutFile": "layouts/gallery.html",
  "placeholders": [
    { "key": "header", "type": "text", "label": "Заголовок" },
    { "key": "cardText", "type": "richText", "label": "Текст карточки" }
  ],
  "settings": [
    { "key": "sequenceId", "type": "sequence", "label": "Идентификатор последовательности"
},
    { "key": "nextLabel", "type": "text", "label": "Подпись кнопки", "default": "Далее" },
    { "key": "backgroundImage", "type": "image", "label": "Фон страницы" }
  ]
}
```

Типы настроек: `number`, `boolean`, `select`, `text`, `image`, `sequence`. Перечень закрыт — неизвестный тип не пройдёт валидацию.

Настройка может объявить `default` (подставляется при создании страницы) и `required` (незаполненная настройка блокирует сохранение так же, как обязательный placeholder). Редактор структуры собственных полей не имеет: форма страницы — это ровно то, что объявил вариант.

`sequence` — последовательность страниц. Подряд идущие страницы одного участка структуры с одинаковым непустым значением образуют последовательность. Число точек индикатора и текущую позицию считает ядро и передаёт макету через `page.*`; автор их не вводит. Требуется минимум две страницы: отрезок из одной точек не даёт. Вариант, не объявивший `sequence`, поля автору не показывает и точек не получает.

Разметка индикатора — дело макета:

```
<div class="gallery__dots">
  {{#each page.dots}}<span class="dot {{statusClass}}"></span>{{/each}}
</div>
{{#if page.canGoBack}}<button data-nav="prev">Назад</button>{{/if}}
<button data-nav="next">{{page.nextLabel}}</button>
```

`nextLabel` — подпись кнопки «вперёд». Ядро отдаёт её макету как `page.nextLabel` (со значением по умолчанию «Далее», если автор поле не заполнил). Кнопку помечают атрибутом `data-nav="next"` — по нему её находят оба рантайма; класс обёртки роли не играет.

`backgroundImage` **— фон страницы.** Свойство, а не содержимое: ядро применяет его к корневому элементу экрана напрямую, макету ничего описывать не нужно.

`sectionSubtitleShown` **и** `sectionSubtitle` **— подзаголовок раздела.** Пара настроек варианта вида `intro`: строка над авторской инструкцией на экране «Введение раздела». Объявляется так:

```
"settings": [
  { "key": "sectionSubtitleShown", "type": "boolean", "default": true,
    "label": "Показывать подзаголовок раздела" },
  { "key": "sectionSubtitle", "type": "text", "default": "Инструкция",
    "label": "Подзаголовок раздела", "maxLength": 120 }
]
```

Разрешённую строку ядро отдаёт макету как `page.sectionSubtitle`. Печатать её надо ПО УСЛОВИЮ, а блок инструкции при этом сохранять:

```
{{#if sectionIntro.hasInstruction}}
<div class="tb-card">
  {{#if page.sectionSubtitle}}<span class="tb-card__head">{{ page.sectionSubtitle }}</span>
{{/if}}
  <span class="tb-card__body" data-slot="instruction"></span>
</div>
{{/if}}
```

Правило пустоты здесь ОБРАТНОЕ привычному по `nextLabel`. Пустая строка приходит только от выключенного тумблера — так автор убирает надпись. Пустое ПОЛЕ при включённом тумблере значит «следовать шаблону» и печатает умолчание: контрол настройки показывает умолчание серой подсказкой, поэтому «не трогал» и «очистил» неотличимы в форме, и пустота не может означать выключение. Макет, напечатавший надпись безусловно, отнимет у автора возможность её убрать.

`passConditionShown` **— строка условия прохождения.** Тумблер варианта вида `intro`, с версии 3.15.0: убирает строку «Для прохождения: 7 баллов из 10» на конкретном введении раздела.

```
{ "key": "passConditionShown", "type": "boolean", "default": true,
  "label": "Показывать условие прохождения" }
```

Макету ничего связывать не нужно: выключенный тумблер ядро превращает в пустой `sectionIntro.passCondition`, и строку убирает тот же гейт, что и раньше. Печатать условие надо только под ним:

```
{{#if sectionIntro.passCondition}}<span class="tb-scene__lead" data-
path="sectionIntro.passCondition"></span>{{/if}}
```

Тумблер меняет только надпись. Порог темы и вердикт остаются прежними, а метка «Обязательная тема» (`sectionIntro.isRequired`) и предупреждение о таймере от него не зависят.

8.1.1 Эталонная сетка вариантов контентной страницы

Оба поставляемых шаблона несут одну и ту же сетку из шести раскладок в двух семействах — обычная страница (`info.*`) и галерейная (`gallery.*` , с индикатором последовательности и «Назад»). Это не нормативный контракт, а образец: свой шаблон вправе объявить меньше или больше.

Раскладка	Ключи	Поля
Текст	`info.text` / `gallery.text`	Заголовок (`text`) + Текст (`html`)
Текст с подзаголовком	`info.text-lead` / `gallery.text-lead`	+ Подзаголовок (`text`)
Текст, изображение слева	`info.image-left` / `gallery.image-left`	Текст + Изображение (50% ширины)
Текст с подзаголовком, изображение слева	`info.image-left-lead` / `gallery.image-left-lead`	+ Подзаголовок + Изображение
Текст, изображение справа	`info.image-right` / `gallery.image-right`	Текст + Изображение
Текст с подзаголовком, изображение справа	`info.image-right-lead` / `gallery.image-right-lead`	+ Подзаголовок + Изображение

Принципы сетки:

- поле текста объявлено типом `html` — это ровно три режима ввода: простой текст, форматированный, HTML (потолок задаёт тип, §8.2);
- у обычных раскладок настройки — `nextLabel` , `backgroundImage` ; у галерейных добавляется `sequenceId` ;
- одна раскладка «текст с изображением» покрывает и левое, и правое расположение — сторону задаёт порядок блоков в макете, отдельного правила нет;
- один вариант `info.*` помечен `isDefault: true` — на него подменяется страница, чей вариант недоступен (см. ниже);
- демо не раздувается: все галерейные демо-страницы лежат в ОДНОЙ последовательности (общий `sequenceId`), поэтому индикатор виден, а демонстрации по одной на вариант достаточно.

8.1.2 Недоступный вариант и `isDefault`

Автор может выбрать другой шаблон оформления теста, и тогда часть страниц окажется привязана к вариантам, которых новый шаблон не объявляет. Такая страница НЕ ломается: она рендерится вариантом по умолчанию своего `kind`. Вариант по умолчанию — тот, что помечен `isDefault: true`; если пометки нет, берётся ПЕРВЫЙ объявленный вариант этого `kind`. Поэтому у каждого `kind`, который может остаться без привязки, полезно назначить осмысленный вариант по умолчанию.

Подмена — решение отрисовки, а не правки данных: `templateKey` страницы в базе не меняется, привязка переживает возврат прежнего шаблона, а «Структура» продолжает помечать страницу как требующую сопоставления. Окончательную замену подтверждает автор в диалоге «Сменить вариант» (там же видны теряемые значения). Если шаблон не объявляет НИ ОДНОГО варианта нужного `kind`, подменять нечем — страница деградирует до простого макета (заголовок и текст). Правило едино для рантайма пакета, веб-хоста и предпросмотра (`shared/template/content-page.ts`).

8.2 Какой HTML допустим в полях контента

Разметку принимают только два типа полей — `richText` и `html`. Значения остальных типов экранируются при отрисовке, поэтому теги в них выводятся как текст.

Значение проходит санитизацию ДВАЖДЫ: при сохранении страницы (автор сразу видит в форме предупреждение со списком удалённого) и повторно при сборке SCORM-пакета. Санитизация серверная; рендерер вставляет `richText` / `html` в страницу без дополнительной обработки. Кроме удаления небезопасного, санитайзер ограничивает авторский CSS блоком поля — см. «Стили внутри значения поля» ниже.

Что удаляется всегда:

Что	Правило
`<script>…</script>`	Вырезается вместе с содержимым
`<iframe>…</iframe>`	Вырезается вместе с содержимым
`<svg>…</svg>`	Вырезается вместе с содержимым (инлайновый SVG в контенте не поддерживается)
`<object>`, `<embed>`, `<link>`, `<meta>`	Вырезаются и парные, и одиночные формы
Атрибуты-обработчики `on*`	`onclick`, `onmouseover`, `onerror` и любые другие удаляются
`href="javascript:…"`, `src="javascript:…"`	Значение заменяется на `#`
`src` на внешний `http(s)://`	Атрибут удаляется целиком — на любом теге
`href` на внешний `http(s)://` у тега, кроме `<a>` и `<area>`	Атрибут удаляется целиком (в том числе у `<base>`)

Последние два правила — про автономность пакета, но она касается не всякого адреса. `src` называет РЕСУРС, который страница грузит сама: внешний адрес заставил бы SCORM-курс ходить в интернет и выдал бы адрес учащегося третьей стороне. `href` у ссылки — НАПРАВЛЕНИЕ, по которому учащийся идёт сам, и до клика не запрашивается ничего, поэтому такой адрес сохраняется: `<a href="https://example.com">Тут</a>` остаётся ссылкой. Внутренние ссылки (`/uploads/media/…`, относительные пути, якоря `#…`) остаются, как и раньше.

Что дописывается: внешняя ссылка получает `target="_blank" rel="noopener noreferrer"`. Внутри LMS пакет живёт в кадре, и переход в том же кадре увёл бы учащегося с незавершённой попытки. Заданные автором `target` и `rel` не перекрываются, а повторная санитизация сохранённого значения ничего не меняет. Внутренние и почтовые (`mailto:`) ссылки атрибутов не получают.

Что обязан сделать шаблон. Оформить ссылку: `<a>` без адреса браузер не отличал от текста, и правила для неё в шаблоне могло не быть, а браузерное умолчание не знает палитры. Правило объявляется на ОБЛАСТИ авторской разметки, а не на `a` вообще — ядро печатает собственные ссылки (фишки рекомендаций `.tb-rec`), и общее `a[href]` перебило бы их по специфичности. Поставляемые шаблоны делают так:

```
/* theme.css — экраны */
.tb-cover__desc a,
.tb-content__lead a,
.tb-content__body a,
.content-placeholder a,
.tb-scene__note a,
.tb-scene__qtitle a,
.question-text a,
.tb-intro a,
.tb-recs-group__text a,
.tb-topic-card__fb-text a { color: var(--ou-accent-default); text-decoration: underline; }

/* report.css — документ отчёта: лист у шаблонов разный, поэтому цвет берётся у текста */
.tb-report a { color: inherit; text-decoration: underline; }
```

Что проходит: всё остальное. Форматирование (`<b>` , `<i>` , `<u>` , `<s>` , `<code>`), структура (`<p>` , `<br>` , `<h1>` … `<h6>` , `<ul>` , `<ol>` , `<li>` , `<blockquote>` , `<hr>`), таблицы (`<table>` , `<tr>` , `<td>` …), контейнеры (`<div>` , `<span>`), изображения с внутренним `src` , атрибуты `class` , `id` , `style` , `title` , `alt` , `colspan` и прочие, не перечисленные выше.

Ограничения текущей реализации — знать разработчику шаблона:

- список ЗАПРЕЩЁННОГО, а не разрешённого: любой тег вне таблицы выше пройдёт, включая `<style>` , `<form>` , `<input>` , `<button>` . Инлайновый `<style>` разрешён, но его правила ограничиваются блоком своего поля — см. «Стили внутри значения поля» ниже;
- URI вида `data:` не блокируются;
- санитайзер работает регулярными выражениями, без разбора DOM: намеренно исковерканная разметка теоретически может обойти правило. Поле контента — не граница доверия для авторов-злоумышленников, а защита от случайной вставки небезопасного фрагмента;
- `allowedMarks` и `allowedBlocks` из контракта ИСКЛЮЧЕНЫ: набор средств форматирования одинаков во всех полях и шаблонах, а «что уместно в этом макете» задаётся типом поля (см. режимы ниже). Объявления в старых манифестах ничего не значат — их можно удалить.

Практический вывод: проектируйте макет так, чтобы он выглядел прилично и на голом тексте, и на насыщенной разметке — минимальный набор, на который можно рассчитывать, это абзацы, списки, инлайновое форматирование, заголовки и внутренние ссылки.

Стили внутри значения поля. Автор нередко вставляет в поле не фрагмент, а целый HTML-документ из внешнего редактора — вместе с `<style>` , где есть правила для `body` и `*` . В предпросмотре такая вставка выглядит правильно (веб-хост рендерит экран в Shadow DOM, где селектор `body` не совпадает ни с чем), а в SCORM-пакете и отладчике та же разметка попадает в НАСТОЯЩИЙ документ и переопределяет оформление плеера: шаблон с фиксированной сценой 16:9 схлопывается в пустой экран.

Поэтому селекторы авторского CSS переписываются так, чтобы совпадать только внутри своего блока:

Что написал автор	Что сохраняется
`body`, `html`, `:root`	сам блок поля (`[data-placeholder="<ключ>"]` , у страницы режима HTML — `.content-page--html`)
`*`	`[data-placeholder="…"] *`
`.card`, `h1, h2` и любой другой селектор	тот же селектор с префиксом блока (каждая часть списка отдельно)
`@media`, `@supports`, `@container`	само правило не меняется, вложенные селекторы получают префикс
`@keyframes`, `@font-face`	не меняются
`@import`	удаляется (внешний ресурс ломает автономность пакета)

Обработка идёт в том же санитайзере, то есть во всех трёх точках сразу: при вставке в редакторе, при сохранении страницы и при сборке пакета. Повторный проход ничего не наращивает (обработанный блок помечен атрибутом `data-tb-scoped`), а страницы, сохранённые до появления правила, чинятся сами при следующей сборке. Автор видит в форме отдельную строку предупреждения — сколько правил ограничено и в каком поле.

Чего ограничение НЕ делает — это важно знать разработчику шаблона:

- вьюпортные единицы и `position: fixed` во вставке остаются вьюпортными: переписывается совпадение селектора, а не значение свойства. `min-height: 100vh` внутри блока по-прежнему считается от окна, а не от сцены;
- вставка не начинает масштабироваться вместе со сценой: если макет задаёт типографику контейнерными единицами (`cqh` / `cqw`), а во вставке пиксели, на маленькой сцене пропорции разойдутся. Это вопрос содержания страницы, а не изоляции;
- это по-прежнему не граница доверия: намеренно исковерканный CSS теоретически обойдёт разбор.

Режимы ввода и что видит автор. Текстовое поле имеет переключатель: простой текст, форматированный текст, HTML. Тип поля задаёт ПОТОЛОК: `html` — доступны все три режима, `richText` — простой и форматированный, `text` / `textarea` — только простой. То есть выбором типа шаблон решает, может ли автор вообще вносить разметку в это место макета.

В режиме форматированного текста автор работает кнопками (полужирный, курсив, зачёркнутый, списки, ссылка, очистка форматирования), а вставленный из внешнего источника фрагмент нормализуется — запрещённое вычищается сразу. В режиме HTML автор пишет разметку сам, и запрещённая конструкция не даёт сохранить страницу: показывается ошибка со списком найденного.

Ссылки на ресурсы шаблона. В содержимом можно ссылаться на файлы самого шаблона — изображения, стили — ОТНОСИТЕЛЬНЫМ путём ровно так, как они лежат в ZIP:

```
<img src="images/diagram.png" alt="Схема">
```

То же правило действует и в САМИХ МАКЕТАХ шаблона: макет ссылается на свои файлы так же, как они лежат в архиве, без каких-либо префиксов хоста.

```
<!— layouts/start.html —>
<img class="hero" src="assets/images/hero.png" alt="">
```

Базовый путь подставляет ядро при отрисовке: внутри SCORM-пакета файлы шаблона лежат в `template/` (запасные макеты встроенного шаблона — в `template-default/`), на веб-хосте они раздаются маршрутом `GET /api/templates/<id>/assets/<путь в архиве>`. Подстановка одинакова на всех путях отрисовки — рантайм пакета, экран веб-хоста, предпросмотр шаблона в реестре — поэтому картинка, которая видна в предпросмотре, видна и ученику.

Медиа автора адресуются абсолютным `/uploads/media/…`, а внешний `http(s)`-адрес в `src` удаляется, поэтому относительный путь однозначно означает ресурс шаблона. Документируйте в описании шаблона, какие файлы автор может использовать.

---

9. Параметры (params)

Параметры — это настройки оформления, которые автор теста меняет во вкладке «Оформление». Поддерживаемые типы (`DesignParamType`): `text`, `color`, `boolean`, `select`, `multiselect`, `number`, `url`, `image`, `asset`, `file`, `downloadLink`. Медиа-типы (`image` / `asset` / `file` / `downloadLink`) сериализуются в единый media-конверт. Значения автора пробрасываются:

- в CSS — как переменные (цвета хранятся как HSL-компоненты, см. ниже);
- в `template.js` — через `window.TestBuilder.context.get().params`.

Пример описания параметра:

```
{
  "key": "primaryColor",
  "type": "color",
  "label": "Цвет кнопок и активных элементов",
  "description": "Кнопки «Далее» и «Начать», выбранный вариант ответа, пройденные точки прогресса.",
  "default": "217 91% 42%",
  "group": "Цвета",
  "section": "branding"
}
```

Поле `description` — необязательное; форма «Оформление» выводит его под контролом. Для цвета это важно: подпись называет токен, а не элемент экрана, и без описания автор задаёт «акцентный цвет», не зная, где он проявится. Пишите в описании, ЧТО параметр красит, словами автора теста.

9.0.1 Выбор картинкой: `optionPreviews`

Когда варианты `select` различаются ОБЛИКОМ, список слов не годится: «B2O» ничего не показывает, и автор выбирает вслепую. Объявите картинку на вариант — путь к файлу вашего же пакета, — и редактор нарисует карточки с изображениями:

```
{
  "key": "brandLogo",
  "type": "select",
  "label": "Логотип направления",
  "options": ["plain", "b2b"],
  "optionLabels": { "plain": "Без направления", "b2b": "B2B" },
  "optionPreviews": {
    "plain": "assets/logos/plain.png",
    "b2b": "assets/logos/b2b.png"
  },
  "default": "plain"
}
```

Если картинка под светлый и тёмный интерфейс разная — задайте пару, и редактор возьмёт ту, что подходит теме автора:

```
{
  "optionPreviews": {
    "b2b": { "light": "assets/logos/b2b-light.png", "dark": "assets/logos/b2b-dark.png" }
  }
}
```

Файлы проверяются как любые ссылки манифеста: отсутствующий блокирует импорт, внешний `http(s)`-адрес запрещён, лежащий в пакете не попадёт в «неиспользованные». Превью для варианта вне `options` — предупреждение, картинка игнорируется. Вариант, у которого превью нет вовсе, рисуется карточкой со значком-заглушкой — так и оформляют вариант со смыслом «ничего не показывать». Без `optionPreviews` параметр остаётся обычным списком.

Картинки нормализуются по одному полю и вписываются в него, поэтому пропорции у вариантов могут быть разными — подгонять их под общий размер не нужно.

9.0.2 Значение как атрибут сцены: `dataAttr`

`cssVar` кладёт значение в кастомное свойство, а свойство — только значение: выбрать по нему правило нельзя. Чтобы CSS шаблона МЕНЯЛ картинку по выбранному варианту, объявите атрибут:

```
{ "key": "brandLogo", "type": "select", "dataAttr": "data-brand-logo", "default": "plain" }
```

Оба плеера положат значение в этот атрибут на корень сцены — туда же, где тема (`data-theme`). Дальше выбирайте обычным селектором:

```
[data-brand-logo="b2b"] .tb-scene__logo { background-image: url("…");
}
```

Имя обязано быть вида `data-имя` (строчные латинские буквы, цифры, дефис) — иначе импорт блокируется: произвольное имя хост не пишет, и шаблон молча рисовал бы запасной вариант. Нет значения у теста — берётся `default`; нет и его — атрибут не ставится.

Осторожно с темой в вебе: там сцена живёт в теневом дереве, а класс темы висит на его host снаружи, и потомковый селектор до него не достаёт. Правило пишется парой:

```
.ou--dark .tb-scene__logo,
:host(.ou--dark) .tb-scene__logo { … }
```

9.1 Цвета стандартного шаблона: где какой виден

Восемь цветовых параметров поставляемых шаблонов названы и упорядочены по фактическому применению в `base.css` — по значимости, от того, что автор меняет чаще всего:

Параметр	Где виден
Цвет кнопок и активных элементов (`primaryColor`)	Кнопки «Далее»/«Начать», выбранный вариант ответа, пройденные точки прогресса, рамка фокуса
Фон экрана (`backgroundColor`)	Подложка всего экрана под карточками и текстом
Цвет основного текста (`foregroundColor`)	Заголовки, текст вопроса, текст учебных страниц
Фон карточки (`cardColor`)	Заливка карточек: учебная страница, карточка вопроса, окно подтверждения, итоги
Граница карточки (`cardBorderColor`)	Контур карточек и модальных окон
Границы элементов (`borderColor`)	Рамки вариантов ответа и полей, разделители внутри карточек
Фон вспомогательных блоков (`mutedColor`)	Невыполненные точки прогресса, вставки с пояснением, дорожки шкал
Подсветка при наведении (`accentColor`)	Заливка варианта ответа и ссылки под курсором мыши

Помимо цветов, секция `branding` несёт media-параметр `logoUrl` (тип `image`) — логотип в шапке старта и итогов (`design.logoUrl`, §7.5); он необязателен (`default: null`). Иллюстрация стартового экрана параметром оформления НЕ задаётся: это свойство страницы у варианта старта (§4).

---

10. Темизация: theme.css и base.css

Токены оформления объявляются в `theme.css` на `:root`. **Цвета хранятся как HSL-компоненты** (без `hsl( … )`), чтобы их можно было переопределять параметрами и собирать в `hsl(var(--token))`:

```
:root {
  --background: 225 7% 7%;
  --foreground: 0 0% 98%;
  --primary: 217 91% 42%;
  --card: 225 14% 14%;
  --border: 0 0% 22%;
  --font-sans: Inter, -apple-system, 'Segoe UI', sans-serif;
  --radius: 12px;
}
```

В `base.css` используйте токены через `hsl( … )`:

```
.start-title {
  color: hsl(var(--foreground));
  font-family: var(--font-sans);
}
.btn {
  background: hsl(var(--primary));
  border-radius: var(--radius);
}
```

Изоляция: в веб-предпросмотре шаблон монтируется в Shadow DOM, а селекторы `:root` / `body` отображаются на корень тени. Не полагайтесь на глобальные стили страницы-хоста — всё нужное объявляйте в своих CSS.

10.1. Две палитры: светлая и тёмная

Шаблон может поставлять две палитры. Базовая объявляется на `:root`, вторая — в блоке `prefers-color-scheme` и в явном переопределении по атрибуту:

```
:root,
:root[data-theme="light"] { --background: 240 4% 93%; --primary: 15 100% 45%; }

@media (prefers-color-scheme: dark) {
  :root:not([data-theme="light"]) { --background: 240 10% 10%; --primary: 15 100% 52%; }
}

:root[data-theme="dark"] { --background: 240 10% 10%; --primary: 15 100% 52%; }
```

Одних стилей мало: платформа узнаёт о палитрах ТОЛЬКО из манифеста. Объявите их в `themes[]` — иначе автор теста не сможет ни выбрать вид теста, ни задать цвет для второй палитры, а валидатор при загрузке предупредит об этом:

```
"themes": [
  { "id": "light", "label": "Светлая" },
  { "id": "dark",  "label": "Тёмная" }
]
```

Перечень идентификаторов закрыт: `light`, `dark`. Заявленная тема обязана объявлять хотя бы один свой токен в стилях — иначе загрузка и активация отклоняются: автор получил бы колонку цветов, которая ничего не красит.

Что делает платформа дальше:

- закреплённый автором вид теста ставится на корень сцены атрибутом `data-theme` (`<html>` в пакете, shadow-host на вебе); при «Авто» атрибут не ставится и работает ваш медиа-запрос;
- цветовые переопределения теста печатаются отдельным `<style>` ПОСЛЕ ваших стилей, правилами той же структуры — на равной специфичности побеждает порядок.

Не прибивайте фирменный цвет мимо токена. Декоративная плашка с фиксированным градиентом переживёт смену темы, но разойдётся с остальным экраном, как только автор сменит основной цвет; ведите такие заливки от `var(--primary)`.

---

11. Браузерный скрипт template.js

Опциональный lifecycle-скрипт. Доступен глобальный объект `window.TestBuilder`:

API	Назначение
`TestBuilder.template.on(event, cb)`	Подписка на событие жизненного цикла
`TestBuilder.template.emit(event, data)`	Отправка события
`TestBuilder.context.get()`	`{ params }` — эффективные значения параметров
`TestBuilder.scorm.commit()`	Принудительная фиксация SCORM-состояния
`TestBuilder.ui.toast(msg)` / `ui.modal(opts)`	UI-хелперы — сейчас заглушки: только `console.warn`, реального тоста/модалки не рендерят

Событие `page:enter` (с полезной нагрузкой `{ page }`) эмитится при рендере контентной страницы; на экранах вопроса, старта и результатов оно не срабатывает. Событийная шина также эмитит события маршрутизатора: `router:shown`, `router:sectionSelected`, `router:finalResultUnlocked`, `router:finalResultOpened`. Минимальный скрипт, обновляющий полосу прогресса:

```
/**
 * @module template
 * @description Updates the progress fill on page transitions.
 */
(function () {
  "use strict";
  var tb = window.TestBuilder;
  if (!tb) return;

  function updateProgress() {
    var fill = document.getElementById("tb-progress-fill");
    if (!fill) return;
    try {
      fill.style.width = (TEST_DATA.progress.question.percent || 0) + "%";
    } catch (_) {}
  }

  tb.template.on("page:enter", function () {
    updateProgress();
  });

  document.addEventListener("DOMContentLoaded", updateProgress);
})();
```

Ограничения скрипта:

- Только браузерный код. Никаких `import` / `require` / `fetch` с внешних адресов — это блокирует активацию.
- Скрипт проверяется только компиляцией (синтаксис), но не исполняется на сервере. Ошибка времени выполнения проявится при рендере экрана в проверке.

---

12. Демонстрационный набор данных (demo/course.json)

`preview.demoData` указывает на JSON с данными, на которых строятся живой предпросмотр и проверка работоспособности. Каждый маршрут из `preview.routes` должен иметь соответствующие данные. Структура:

```
{
  "schemaVersion": "1.0",
  "locale": "ru-RU",
  "params": { "primaryColor": "217 91% 42%", "fontFamily": "Inter" },
  "course": {
    "title": "Основы информационной безопасности",
    "description": "Демонстрационный тест",
    "passPercent": 70,
    "timeLimitMinutes": 30,
    "maxAttempts": 3,
    "questionCount": 4,
    "topics": [
      { "id": "topic-1", "title": "Базовые угрозы", "status": "available" }
    ],
    "contentPages": [
      {
        "id": "demo-intro", "type": "intro", "route": "content.intro",
        "templateKey": "intro.hero",
        "values": { "title": "Введение", "subtitle": "Читайте внимательно", "heroImage":
null }
      }
    ],
    "questions": [
      {
        "id": "demo-single", "type": "single",
        "prompt": "Какой пароль самый надёжный?",
        "options": [
          { "id": "s1", "text": "qwerty123" },
          { "id": "s2", "text": "Длинная случайная фраза", "correct": true }
        ]
      },
      {
        "id": "demo-matching", "type": "matching", "prompt": "Сопоставьте угрозу и
описание.",
        "pairs": [ { "id": "p1", "left": "Фишинг", "right": "Поддельное письмо" } ]
      },
      {
        "id": "demo-ranking", "type": "ranking", "prompt": "Расставьте шаги по порядку.",
        "options": [ { "id": "r1", "text": "Обнаружить" }, { "id": "r2", "text": "Сообщить"
} ],
        "order": ["r1", "r2"]
      }
```

```
    ]
  },
  "runtime": {
    "route": "start",
    "progress": {
      "active": { "current": 2, "total": 4, "percent": 50 },
      "question": { "current": 2, "total": 4, "percent": 50 },
      "page": { "current": 4, "total": 9, "percent": 44 }
    },
    "result": {
      "scorePercent": 86, "passed": true, "status": "passed",
      "totalQuestions": 4, "correct": 3, "earnedPoints": "8.6",
      "passClass": "is-pass", "statusLabel": "Пройден", "ringDashoffset": "55.42",
      "topicResults": [
        { "topicId": "topic-1", "topicName": "Базовые угрозы",
          "percent": 75, "passed": true, "total": 4, "correct": 3,
          "passClass": "is-pass", "statusLabel": "Пройден" }
      ]
    },
    "sectionResult": { "scorePercent": 75, "status": "passed" }
  }
}
```

Соответствие маршрут → данные:

- `start`, `results` — берут `course.*` и `runtime.result` (итог всего теста);
- `content.summary` (легаси «Итог раздела») — берёт `runtime.sectionResult` (результат раздела) в `result.*`;
- `section-results` — берёт `runtime.sectionResult` в `sectionResult.*`; `review` — строит `review.*` из неотвеченных вопросов;
- `question.<type>` — ищут вопрос по `questionId` из маршрута, иначе первый вопрос подходящего типа;
- `content.<kind>` — берут страницу по `templateKey` / `route` из `contentPages`.

Помимо этого `runtime` может нести `route` (стартовый маршрут предпросмотра), `progress` (значения прогресса; доступны в `template.js` как `TEST_DATA.progress.*`) и `state` (служебное состояние). Их показывает эталонный `demo/course.json`.

---

13. Валидация и проверка работоспособности

Активация шаблона возможна только после двух проверок.

13.1 Структурная валидация (сервер)

Выполняется при загрузке. Блокирующие ошибки (нельзя активировать):

Код	Причина
`MANIFEST_MISSING`	Нет `manifest.json` в корне
`MANIFEST_INVALID_JSON`	Манифест не парсится
`MANIFEST_SCHEMA`	Нарушена схема (нет `id` / `name` / `version` / `contentTemplates` и т.д.)
`ID_PATTERN`	`id` не соответствует `^[a-z0-9-]+$`
`API_VERSION_UNSUPPORTED`	`templateApiVersion` не из поддерживаемых (`1.0`)
`ID_EXISTS` / `ID_MISMATCH`	Конфликт id при создании / несовпадение при обновлении
`REQUIRED_FIELD_MISSING`	Нет `assets.preview` , `preview` , `params` или `capabilities`
`FILE_MISSING`	Файл, на который ссылается манифест, отсутствует
`LAYOUT_TEMPLATE_SYNTAX`	Макет содержит невалидный DSL (незакрытый блок, `{{{ }}}` , выражение) — не компилируется
`EXTERNAL_URL`	Внешняя ссылка/CDN в ресурсах, CSS, HTML или JS
`DEMODATA_INVALID_JSON`	Невалидный JSON демо-данных
`REPORT_VARIANT_INVALID`	Нарушен контракт варианта отчёта (§7.3а) — см. перечень ниже
`ZIP_TOO_LARGE`	Архив больше лимита (по умолчанию 20 МБ)

Блокирует только то, при чём шаблон нечитаем или небезопасен. Отсутствие макета или слота активацию **не** блокирует.

`REPORT_VARIANT_INVALID` выдаётся, когда вариант отчёта: не объявил `layoutFile` ; сослался на отсутствующий в архиве макет или `styleFile` ; объявил поле типа `sequence` или непустой `placeholders[]` ; не имеет ровно одного `isDefault` на свой вид; отдал макет без корневого класса `tb-report` или с классами слоя сцены (`tb-scene*`); отдал `styleFile` с селектором вне `.tb-report` либо адресующим документ (`:root` , `html` , `body`). Сообщение называет вариант и место замечания. Проверка идёт и при загрузке (там видны файлы архива), и при активации (там проверяется только объявление).

Отдельно проверяются ТИПЫ полей: неизвестный тип в `placeholders[]` или `settings[]` , а также несовместимые атрибуты типа (например `select` без списка вариантов) блокируют и загрузку, и активацию; сообщение называет ключ варианта и ключ поля. Шаблон, установленный до введения проверки, продолжает работать, но помечается невалидным, а страница с полем неизвестного типа показывает автору теста диагностику вместо подмены контрола.

Предупреждения (не блокируют): нет `data-slot="page"` в оболочке (`SHELL_CONTRACT`), нет `question-text` / `question-interaction` в макете вопроса (`QUESTION_CONTRACT`), нет `data-slot="page-content"` в контенте (`CONTENT_CONTRACT`), не объявлен макет экрана, файлы, не используемые манифестом. Во всех этих случаях экран отрисуется из стандартного шаблона.

13.2 Проверка работоспособности (браузер)

Запускается из окна предпросмотра. Движок рендерит каждый экран из `preview.routes` на демо-данных через общий рендерер. Экран помечается проваленным при: исключении рендера, пустом результате, `console.error` во время рендера. `console.warn` и незаполненный слот дают неблокирующее предупреждение. Дополнительно проверяется `template.js` (компиляция).

Сверх маршрутов `preview.routes` проверка рендерит ОТЧЁТ — по одному варианту каждого объявленного вида, на исходе «не пройден» (на нём страница показывает больше: вердикт, непройденные темы, рекомендации). Отчёта в `preview.routes` нет и объявлять его там не нужно: проверка находит варианты сама, по манифесту. Ошибка отрисовки отчёта блокирует активацию наравне с ошибкой экрана — иначе сломанный отчёт всплыл бы только у ученика, который не смог скачать PDF.

---

14. Жизненный цикл в системе

Реестр шаблонов — страница `/author/templates` , пункт «Шаблоны» левого меню (роли «разработчик» и «администратор»):

1. **Загрузить шаблон** — выбрать ZIP. Идёт структурная валидация; при успехе создаётся черновик.
2. **Предпросмотр и проверка** — открыть карточку, осмотреть экраны слева, нажать «Проверить работоспособность». При успехе разблокируется активация.
3. **Активировать** — шаблон появляется у авторов во вкладке «Оформление».
4. **Деактивировать** — шаблон скрывается; зависимые тесты переключаются на «Стандартный» (совместимые параметры сохраняются).
5. **Обновить** — загрузить новую версию (id должен совпадать); требуется повторная проверка.
6. **Экспорт ZIP** — скачать любой шаблон (в т.ч. встроенный) как стартовую заготовку.
7. **Удалить** — только загруженный, неактивный и не используемый тестами.

---

15. Чек-лист перед загрузкой

- [ ] `manifest.json` в корне, валидный JSON.
- [ ] `id` в нижнем регистре (`^[a-z0-9-]+$`), `version` вида `1.0.0` , `templateApiVersion: "1.0"` .
- [ ] Объявлены `layouts.shell/question/content/results` ; все referenced-файлы на месте.
- [ ] В `shell.html` есть `data-slot="page"` .
- [ ] В `question.html` есть `data-slot="question-text"` и `data-slot="question-interaction"` .

- [ ] `assets.preview` указывает на существующую миниатюру.
- [ ] `contentTemplates` содержит минимум один элемент.
- [ ] Типы полей — из закрытых перечней: `placeholders[]` (§8) и `settings[]` (§8.1); у `select` заполнен список вариантов.
- [ ] Нет внешних ссылок/CDN в CSS, HTML, JS и в путях манифеста.
- [ ] `preview.demoData` валиден и покрывает все `preview.routes`.
- [ ] Если шаблон поставляет вторую палитру — она объявлена в `themes[]` (§10.1), и у каждой заявленной темы есть свои токены в стилях.
- [ ] Фирменные заливки ведутся от `var(--primary)`, а не от фиксированного цвета.
- [ ] Если шаблон поставляет отчёт (§7.3a): у каждого варианта есть `layoutFile`, на каждый вид ровно один `isDefault`, в макете корневой класс `tb-report` и ни одного `tb-scene*`, а все селекторы `styleFile` вложены в `.tb-report` (без `:root` / `html` / `body`).
- [ ] Картинки отчёта (подложка, логотип) лежат в пакете, перечислены в `assets.images` и объявлены полями типа `image` с `default` — путём внутри шаблона (§7.3a).
- [ ] Рекомендованные ссылки в отчёте оформлены как `pdf-link-btn` с `data-url` — иначе в PDF они станут картинкой.
- [ ] Если объявлены надписи (§3.4): у каждой есть `default`, ключи уникальны и не являются взаимными префиксами, а макеты печатают их через `{{#if labels.<ключ>}}` — включая макеты отчёта, иначе автору эти надписи для документа не предложат.
- [ ] Экран итогов построен обходом `result.blocks[]`, а не четырьмя жёсткими секциями (§3.5), и печатает вводный блок `result.introHtml` через `{{& … }}`.
- [ ] Карточка темы печатает разрез `{{#if breakdown}}` — на экране итогов и в отчёте одинаково (§7.3); не объявили — автор включит показ и не увидит ничего.
- [ ] Блоки разделов: макет печатает `topicGroups` + `ungroupedTopics` ЛИБО плоский `topicResults`, но не оба сразу (§7.3) — иначе тема выйдет дважды.
- [ ] Сводный разрез объявлен в `resultsBlockOrder` (в том числе для `results.adaptive`) и напечатан по `isBreakdown` (§3.5, §7.3).
- [ ] DSL без `{{{ }}}` и выражений; все блоки закрыты.
- [ ] Архив меньше 20 МБ.

16. Типичные ошибки

Симптом	Причина и решение
`SHELL_CONTRACT`	В оболочке забыт `data-slot="page"`. Добавьте контейнер страницы.
`QUESTION_CONTRACT`	В макете вопроса нет `question-text` или `question-interaction`. Сверьтесь с разделом 4.2.
`EXTERNAL_URL`	Где-то осталась ссылка на CDN (шрифт, скрипт, картинка). Вендорьте ресурс в ZIP.
`THEME_INVALID`	В `themes[]` идентификатор вне перечня `light` / `dark`, тема продублирована, не задан `label` либо у объявленной темы нет ни одного токена в стилях.
`THEME_ADVISORY`	В стилях есть тёмная палитра, но `themes[]` не объявлен — автор её не увидит. Объявите темы (§10.1).
Экран «отрисован пустым» в проверке	Макет ничего не вывел или DSL-условие скрыло весь контент. Проверьте имена путей контекста.
«Ошибка отрисовки» в проверке	Невалидный DSL: незакрытый `{{#if}}` / `{{#each}}`, `{{{ }}}` или выражение с пробелом.
Не заполнен слот в проверке	Имя `data-slot` не совпадает с ожидаемым (`question-text`, `question-interaction`, `page-content`).
Экран отрисовался чужим оформлением	Шаблон не объявил этот экран, и сработал fallback на стандартный. Проверьте `layoutFile` варианта: ключ `layouts["content.intro"]` ядро не читает (раздел 4.3.1).
Значения автора не выводятся	Placeholders варианта не совпадают с `data-placeholder` в макете (раздел 4.3.1).
Шаблон не принимается: неизвестный тип поля	Тип вне закрытых перечней (§8, §8.1) — например `richtext` вместо `richText` или `number` в `placeholders[]` вместо `settings[]`. Сообщение называет вариант и поле.
Индикатор последовательности не появляется	Вариант не объявил настройку типа `sequence`, значение не задано автором либо подряд идёт всего одна страница с этим значением — точки требуют минимум двух (§8.1).
Цвета «не подхватываются»	Токен задан как `hsl( ... )` вместо HSL-компонентов; используйте `--token: H S% L%` и `hsl(var(--token))`.
Превью недоступно	Нет `assets.preview` или файл отсутствует в архиве.
`REPORT_VARIANT_INVALID`	Нарушен контракт варианта отчёта: нет `layoutFile`, не тот `isDefault`, запрещённый тип поля, класс `tb-report` или скоуп CSS (§7.3a, §13.1). Сообщение называет вариант и место.

Симптом	Причина и решение
Отчёт красив в LMS и «поехал» в браузере	Макет оперся на `theme.css` или на слой сцены. В пакете CSS шаблона лежит в главном документе, на вебе — внутри Shadow DOM экрана, а отчёт рендерится вне сцены. Тема приходит переменными на контейнер; собственные стили — только в `styleFile` (§7.3а).
В PDF пропал фон или логотип	Картинка не прочиталась к моменту растеризации, и значение поля стало пустым. Проверьте, что файл лежит в архиве, объявлен в `assets.images`, а `default` поля указывает на него относительным путём от корня шаблона (`assets/report/bg.png`, без ведущего слеша).
Подложка есть в пакете, но не появляется на вебе	Макет читает не то поле: картинки приходят только через `report.values.<ключ>`. Полей `report.backgroundUrl` / `report.logoUrl` в контексте нет — их убрали вместе с ассетами ядра.
Ссылки в PDF не нажимаются	Растр сам по себе ссылок не несёт. Чип без `<a>` должен нести класс `pdf-link-btn` и атрибут `data-url`; у `<a href>` адрес должен начинаться с `http(s)://` или `mailto:` — якорь `#…` и относительный путь в PDF не переносятся.
Отчёт оформлен чужим шаблоном	Ваш шаблон не объявил нужный вид (`report` или `report.adaptive`), сработала деградация на «Стандартный». Цвета при этом остаются вашими.
`LABELS_INVALID`	В `labels[]` повторился `key`, у надписи нет `default` либо два ключа оказались взаимными префиксами (`results` и `results.heading`). Контекст резолвит путь деревом, поэтому префиксный ключ невозможен (§3.4).
Автор включил надпись, а в документе её нет	Макет отчёта не печатает её прямым путём `labels.<ключ>`. Перечень надписей отчёта считается по макетам: не печатаете — не предложат, напечатали — предложат (§7.3а).
Автор включил показ разреза, а полос нет	Макет не печатает `topicResults[].breakdown[]` — блока в вёрстке просто нет. Второй возможный случай: у вопросов темы нет ни одного ключа, тогда поля в контексте не будет и у эталонного шаблона (§7.3).
Разрыв страницы «не сработал»	Метка проигнорирована по правилу: она стоит в начале листа, сразу после другого разрыва или после последнего содержимого — пустых листов механика не порождает (§7.3а).